

آموزش صفر تا صد - نسخه ۲۰۲۶

WordPress

صفر تا Headless با Next.js

وردپرس به عنوان بک‌اند؛ Next.js به عنوان فرانت تو

WPGraphQL

REST API

CPT + ACF

WordPress 6.9

Deploy Vercel

Next.js App Router

۱۳ فصل · پروژه کامل Headless · تمرین با جواب · چیت‌شیت و گلاساری

فهرست مطالب

نقشه دوره و مسیر دو نیمه‌ای	شروع دوره	1
CMS، معماری، اکوسیستم، Headless چيست	فصل ۱ – وردپرس چيست	2
برنامه Local، اولین سایت، WP-CLI	فصل ۲ – نصب لوکال با Local	3
پست/صفحه، بلوک‌ها، رسانه	فصل ۳ – داشبورد و محتوا	4
Block Themes، Site Editor، تم فرزند	فصل ۴ – تم‌ها و Site Editor	5
Yoast، نصب، ضروری‌ها، امنیت	فصل ۵ – پلاگین‌ها	6
نقش‌ها، Permalinks، تنظیمات	فصل ۶ – کاربران و تنظیمات	7
Headless – Custom Post Type + ACF – پایه	فصل ۷ – ACF و CPT ★	8
PHP سریع، The Loop، Hooks، اولین پلاگین	فصل ۸ – PHP برای دولوپر JS	9
Application Password، فیلترها، endpoint	فصل ۹ – WP REST API	10
GraphQL در ACF، کوئری‌های دقیق، GraphQL	فصل ۱۰ – WPGraphQL ★	11
CPT، پروژه، ISR، [slug]، لیست	فصل ۱۱ – Next.js + وردپرس 🚀	12
SEO، Vercel، فرم‌ها، Preview	فصل ۱۲ – Headless پیشرفته	13
مرور سریع + گیرهای کلاسیک	فصل ۱۳ – چیت‌شیت و گلاساری	14

دوره جامع WordPress – از صفر مطلق تا Headless با

Next.js

نسخه دوره: ۲۰۲۶ – سازگار با WordPress 6.9.x (با اشاره به 7.0 در راه)، Block Themes/Site Editor و استک Headless مدرن: REST API + WPGraphQL + ACF مخاطب: دولوپر Next.js که از وردپرس هیچی بلد نیست – و در پایان می‌خواهد Next.js رو فرانت و وردپرس رو بک‌اند کنه (Headless)

چرا وردپرس – و چرا برای تو جالبه؟

دو دلیل که این دوره از نظر شغلی برای تو ارزش داره:

- 1. بازار کار عظیم:** ۴۰٪+ کل وبسایت‌های دنیا وردپرسی‌ان. مشتری‌ها و شرکت‌های ایرانی هم عمدتاً وردپرس می‌خوان – و اکثر دولوپرهای وردپرس، Next.js بلد نیستن. ترکیب دوتاش یعنی مزیتی که خیلی‌ها ندارن.
 - 2. Headless = جمع دورهمی مهارت‌ها:** وردپرس فقط پنل مدیریت محتوا می‌شه (رابط کاربری برای مشتری که خودش محتوایش رو وارد کنه) و فرانت رو با همون Next.js می‌سازی که بلدی. مشتری خوشحاله (پنل راحت داره)، تو خوشحالی (React می‌نویسی).
- مسیر دوره دو نیمه داره: - **نیمه اول (فصل ۱-۸):** وردپرس استاندارد – نصب، محتوا، تم، پلاگین، CPT/ACF و کمی PHP - **نیمه دوم (فصل ۹-۱۲):** Headless – REST API، WPGraphQL، و اتصال کامل به Next.js با پروژه واقعی

فصل	موضوع	چی یاد می‌گیری
فصل ۱	وردپرس چیست؟	CMS، معماری، اکوسیستم، wp.org vs wp.com
فصل ۲	نصب لوکال	برنامه Local، اولین سایت، ورود به پنل
فصل ۳	داشبورد و محتوا	پست/صفحه، دسته‌بندی، رسانه، ادیتور بلوکی
فصل ۴	تم‌ها	Block Themes، Site Editor، تم فرزند
فصل ۵	پلاگین‌ها	اکوسیستم، پلاگین‌های ضروری، آناتومی
فصل ۶	کاربران و تنظیمات	نقش‌ها، Permalinks (مهم برای API!)، تنظیمات کلیدی
فصل ۷	CPT و Custom Fields ★	محتوای ساخت‌یافته با ACF – پایه Headless
فصل ۸	PHP برای دولوپر JS	سینتکس سریع، The Loop، Hooks، اولین پلاگین خودت
فصل ۹	WP REST API	curl، endpoint ها، Application Passwords
فصل ۱۰	WPGraphQL ★	GraphQL سرور، کوئری‌های دقیق، ACF در GraphQL
فصل ۱۱	Next.js + وردپرس 🚀	پروژه کامل: لیست و صفحه پست با Server Components
فصل ۱۲	Headless پیشرفته	Preview، فرم‌ها، SEO، تصاویر، دپلوی Vercel
فصل ۱۳	چیت‌شیت و گلاساری	مرور سریع + ارورهای رایج + نقشه راه

زمین تمرین

همه‌چیز روی وردپرس **لوکال** (روی سیستم خودت) انجام می‌شه – هیچ چیز خراب شدنش هزینه نداره:

- سایت تست wp-course.local با برنامه **Local** (فصل ۲)

- اپ Next.js جدا: wp-headless (فصل ۱۱)

قانون طلایی دوره: وردپرس رو مثل یک محصول آماده / استفاده کن (فصل ۲-۶) و مثل یک فریم‌ورک توسعه بده (فصل ۷-۱۲) – دولوپری که هردو رو بلده، در تیم‌های Headless می‌درخشه.

ابزار	وضعیت تو	چک کردن
ویندوز 10/11	✅	—
Next.js 15+ (App Router)	✅ از پروژه‌هایت	<code>npx next --version</code>
Git/GitHub، CLI	✅ سه دوره قبلی	کمک‌کار توی مسیر
PHP / MySQL	❌ لازم نیست بلد باشی	فصل ۲ و ۸ فقط به اندازه لازم

تبدیل به PDF 📄

نسخه PDF رنگی راست‌به‌چپ (فونت وزیرمتن) در همین پوشه آماده است: **«دوره کامل WordPress.pdf»**. برای بازسازی:
+ رندر Chrome headless (دستور در پرامپت آماده پوشه والد). `python build/build_pdf.py`

فصل ۱ – وردپرس چیست و چطور فکرش کنیم؟

🎯 هدف این فصل: مدل ذهنی درست: CMS یعنی چی، وردپرس از چه چیزی ساخته شده، اکوسیستمش چیه، و مهم‌تر از همه – اینکه چرا برای تو به عنوان دولوپر Next.js این ابزار جذابه. هنوز چیزی نصب نمی‌کنیم.

۱.۱ – CMS: وقتی مشتری نمی‌تونه کد بزنه

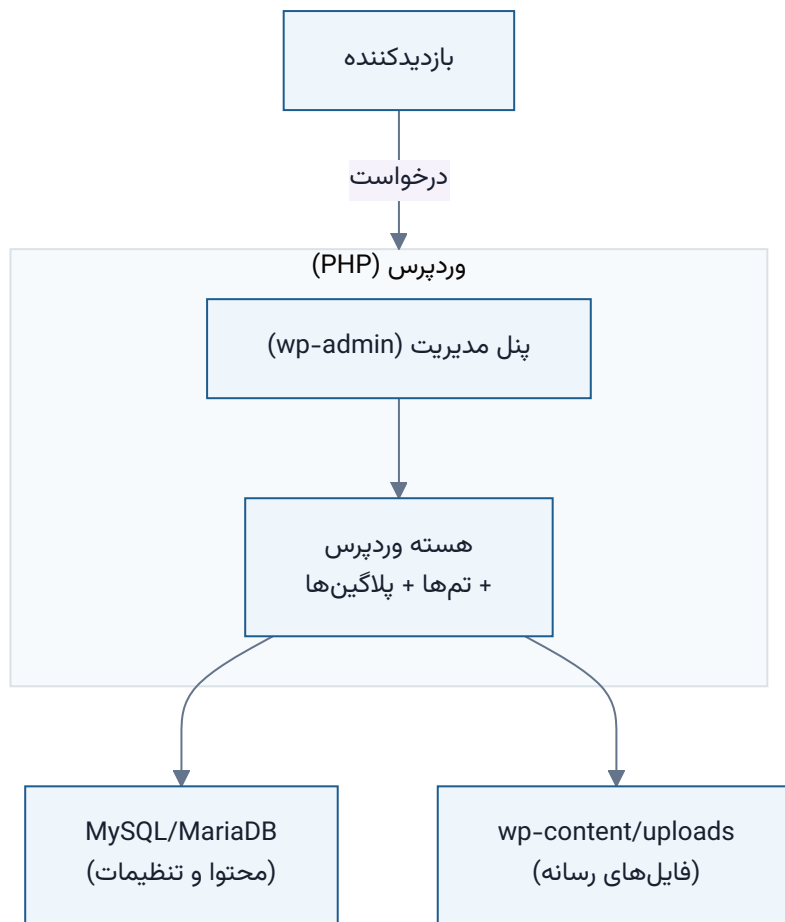
سناریوی آشنا: مشتری یک وب‌سایت می‌خواد که خودش بتونه اخبارش رو وارد کنه. اگه با Next.js خالص بسازی، هر بار مشتری بخواد یک پست بذاره باید بیاد بهت بگه یک commit بزنی! 🤦🏻

CMS (Content Management System) یعنی: یک پنل مدیریت آماده که محتوا رو مشتری اونجا وارد می‌کنه (عنوان، متن، عکس) و سایت ازش می‌خونه. وردپرس = محبوب‌ترین CMS دنیا (نیروی محرکه حدود ۴۰٪+ کل وب؛ بیش از ۶۰٪ سهم بازار CMS).

سایت نمایش می‌دهد → MySQL مشتری → پنل مدیریت وردپرس → محتوا در

۱.۲ – وردپرس زیر hood چی داره؟ (برای تو که مهندسی)

سه لایه:



جزء	چیه	برای تو چه معنایی داره
PHP	زبان سرور وردپرس (۷.۲+)	فقط به اندازه لازم یاد می‌گیری (فصل ۸) – از TS ساده‌تره
MySQL	دیتابیس محتوا	تو SQL بلدی! جدول‌هاش رو فصل ۷ نگاه می‌کنیم
wp-content	جایی که تم‌ها، پلاگین‌ها و آپلودها زندگی می‌کنن	★ تنها جایی که تغییر می‌دی؛ هسته رو هرگز دست نمی‌زنی

۱.۳ – اکوسیستم: تم‌ها و پلاگین‌ها

قدرت واقعی وردپرس، اکوسیستمش است (ده‌ها هزار پلاگین رایگان):

مفهوم	چی می‌کنه	شبیه به
تم (Theme)	ظاهر و قالب سایت – نحوه نمایش	design system + layout
پلاگین (Plugin)	افزودن قابلیت: فروشگاه، فرم، SEO، مولتی‌زبان	npm package
ابزارک/بلوک (Block)	واحدهای UI در ادیتور (پاراگراف، گالری...)	کامپوننت‌های React!

💡 ادیتور وردپرس (گوتنبرگ) از داخل با React نوشته شده! یعنی مفهوم «بلوک» که فصل ۳ می‌بینی، دقیقاً همون کامپوننت‌محوریه که تو می‌فهمی.

۱.۴ – wp.org در برابر wp.com (سوال همیشگی)

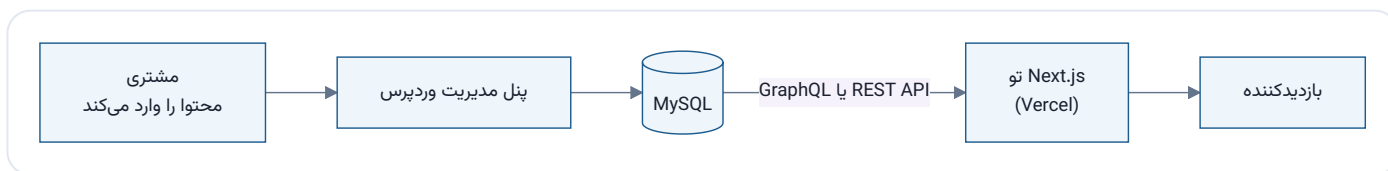
WordPress.com	WordPress.org	
سرویس میزبانی تجاری بر پایه همون	نرم‌افزار متن‌باز – خودت نصب می‌کنی	چیة؟
✗ محدود	✓ کامل	کنترل کامل (تم/پلاگین دلخواه)
✗	✓	نصب لوکال و توسعه
نه	★ این یکی	ما کدوم؟

از این به بعد منظورمون همیشه WordPress.org (خودِ نرم‌افزار) است.

۱.۵ – چرا این دوره برای تو طلایه: Headless

دو راه استفاده از وردپرس:

Headless (بی‌سر!)	Classic (سنتی)	
★ Next.js تو!	تم PHP خود وردپرس	فرانت چیة؟
فقط بک‌اند و پنل محتوا	همه‌چیز: فرانت + بک + پنل	وردپرس چه نقشی داره؟
React/Next (که بلدی!)	PHP	مهارت لازم
★ مدرن SSG/ISR	محدود به PHP	سرعت و UX
همون پنل وردپرس!	پنل وردپرس	برای مشتری



یعنی: مشتری پنل آشنای وردپرس رو می‌گیره، تو فرانت رو با Next.js می‌سازی. بهترین دو دنیا – و دقیقاً همون چیزیه که نیمه دوم دوره می‌سازیش.

نکته واقع‌بینانه: وردپرس Headless برای هر پروژه‌ای نیست (سایت‌های خیلی ساده با تم آماده سریع‌تر ردی می‌شن). ولی برای پروژه‌های جدی با AI سفارشی و پرفورمنس بالا، ترکیب برنده است.

✓ جمع‌بندی فصل

- CMS = پنل محتوا برای مشتری؛ وردپرس = پادشاهش (۴۰٪ وب)
- زیر `wp-content` + PHP + MySQL hood: (تنها جای تغییرات تو)
- تم = ظاهر، پلاگین = قابلیت، بلوک = کامپوننت (با React نوشته شده!)
- WordPress.org (نرم‌افزار) نه. com. (سرویس)
- **Headless**: وردپرس فقط بک‌اند و پنل؛ فرانت = Next.js تو

📝 تمرین فصل ۱

1. با یک جمله: چرا مشتری‌ها CMS می‌خوان ولی دولوپرها همیشه با فریم‌ورک دوستشون ندارن؟
2. در مدل Headless، وردپرس چه لایه‌هایی رو برای تو حل می‌کنه؟ (سه چیز بنویس)
3. فرق wp.org و wp.com رو به یک دوست غیرفنی توضیح بده با analogi مناسب. (مثلاً: اندروید خام در مقابل گوشی با لایسنس...)
4. یک پروژه واقعی فکر کن (مثلاً سایت یک رستوران): کدوم بخش‌هاش محتوا است که مشتری باید خودش ویرایش کنه؟ (منو، آدرس، قیمت‌ها...)

▼ نکته تمرین ۲

پنل مدیریت محتوا (برای مشتری)، REST/GraphQL API (لایه داده استاندارد)، مدیریت کاربران و نقش‌ها و رسانه — یعنی کل بک‌اند CMS. تو فقط UI می‌سازی.

➔ **فصل بعد:** نصب وردپرس روی سیستم خودت با برنامه Local — پنج دقیقه‌ای!

فصل ۲ – نصب لوکال: سایت وردپرسی روی سیستم خودت در ۵ دقیقه

🎯 هدف: وردپرس برای اجرا به سه چیز نیاز داره: PHP + MySQL + وب سرور (همون استک LAMP معروف). به جای نصب دستی، با برنامه Local (محبوبترین ابزار دولوپرهای وردپرس) همه رو یکجا نصب می‌کنیم. آخر فصل، پنل مدیریت تو بالاست!

۲.۱ – راه‌های نصب لوکال (و اینکه چرا Local)

روش	چیه	توصیه
Local (localwp.com)	اپ اختصاصی: PHP+MySQL+سرور یکجا، چند سایت همزمان، CLI و mail catcher	★ استاندارد دولوپرها
Laragon	سبک و سریع ویندوزی	جایگزین خوب
XAMPP	قدیمی‌تر، دستی‌تر	توصیه نمی‌کنم
Docker (وردپرس رسمی)	با دانش دوره داکترت می‌شد!	برای دوره‌های بعدی – Local راحت‌تره

🔗 دانلود: <https://localwp.com> → نسخه Windows. (رایگان؛ ساختاً حساب می‌خواد که فقط ایمیل می‌خواد).
💡 ارتباط با دوره داکر: اگه دقت کنی، Local دقیقاً همون کاری رو می‌کنه که `docker compose up` برای یک استک PHP+MySQL – فقط با GUI. مفهوم «محیط ایزوله هر پروژه» رو حالا از دو زاویه دیدی.

۲.۲ – ساخت اولین سایت، گام به گام

1. Local رو باز کن → دکمه **Create a new site**
2. اسم سایت: `wp-course` → (ادامه)
3. Environment: پیش‌فرض (Preferred) بمون – PHP 8.x + MySQL
4. حساب کاربری وردپرس رو بساز – ★ این اطلاعات رو جایی یادداشت کن: **Username:** `admin` - **Password:** [یک رمز (لوکاله، ساده اشکال نداره) - **Email:** ایمیلی از خودت]
5. صبر کن – Local خودش وردپرس رو نصب می‌کنه (~۱ دقیقه)

۲.۳ – دو دکمه طلایی Local

بعد از اتمام، بالای پنل Local سه دکمه می‌بینی:

دکمه	کار
★ WP Admin	باز کردن پنل مدیریت (wp-course.local/wp-admin) – اینجا خونه‌ی اصلی‌ه
Open Site	باز کردن خود سایت (فرانت)
Start/Stop	روشن/خاموش کردن استک سایت

و دو جاهای مفید پایین پنل:

- **Database tab:** اطلاعات اتصال MySQL (Host/Port/User/Pass) – فصل ۹ برای اتصال از بیرون لازم می‌شه
- **Go to Site Folder / Open Site Shell:** ★ شل داخل محیط سایت – اینجا wp (WP-CLI)، خط فرمان رسمی وردپرس (نصبه! امتحان کن):

```
$ wp core version
6.9.x
```

(WP-CLI = npm برای وردپرس – مدیریت کامل سایت با CLI. در طول دوره بهش برمی‌گردیم.)

۲.۴ – پنل مدیریت: اولین گشت

WP Admin رو بزن → با user/pass خودت لاگین کن. خوش اومدی به wp-admin – یک نوار مشکی کنار، منوهای سمت راست (چون وردپرس RTL رو هم پشتیبانی می‌کنه، اینجا فارسی/راست‌چین می‌بینی):

منو	چی توشه	فصل مربوط
Posts	پست‌های وبلاگ (محتوای زمان‌دار)	۳
Pages	صفحات ثابت (درباره ما، تماس...)	۳
Media	کتابخانه تصاویر و فایل‌ها	۳
Appearance	تم‌ها و Site Editor	۴
Plugins	پلاگین‌ها	۵
Users	کاربران و نقش‌ها	۶
Settings	تنظیمات کلیدی	۶

اولین کار حرفه‌ای: **Site Language** → **General** → **Settings** → **فارسی** – وردپرس کامل فارسی و RTL می‌شه (ترجمه رسمی خودش داره). Timezone هم بذار Tehran.

۲.۵ – سلامت‌های پایه (عادت از همین اول)

- بعد از هر تغییر مهم: در Local یک **Snapshot** بگیر (دکمه بالا کنار اسم سایت) – مثل کامیت گیت! هر وقت خراب شد، Restore.
- در فصل‌های بعد هر وقت وردپرس آپدیت خواست (نوار بالا)، بذار آپدیت شه – نسخه‌های x.6.9 امنیتی‌ان.

✓ جمع‌بندی فصل

- استک وردپرس = MySQL + PHP + وب‌سرور؛ **Local** همه را با GUI نصب می‌کند
- دو دنیا: **WP Admin** (پنل) و خود سایت؛ WP-CLI = Open Site Shell
- فارسی‌سازی از Settings → Site Language
- Snapshot = کامیت محیطی؛ آپدیت‌های امنیتی را نگیر نده

📝 تمرین فصل ۲

1. Local و سایت wp-course را نصب کن و به پنل لاگین کن.
2. زبان را فارسی کن – منوها را دوباره نگاه کن (همه RTL شدند).
3. در Open Site Shell → Local بزن و wp core version و wp plugin list را بزن – چه پلاگین‌هایی از قبل نصب‌اند؟
4. یک Snapshot به اسم «after-install» بگیر.
5. (کشف آزاد) در Local → Database tab، اطلاعات اتصال را ببین و در یک فایل یادداشت ذخیره کن (DB_HOST، DB_USER، DB_PASSWORD، DB_NAME) – فصل ۹ با این‌ها سرور GraphQL/API رو از بیرون تست می‌کنیم.

▼ نکته تمرین ۳

```
wp core version      # 6.9.x
wp plugin list       # فعال/غیرفعال status با - (پلاگین‌های پیش‌فرض) Hello Dolly و Akismet معمولاً
```

WP-CLI از اینجا به بعد دوست خوبته: wp plugin list --status=active و ...

➔ **فصل بعد:** داشبورد و محتوا – پست، صفحه، دسته‌بندی و ادیتور بلوکی گوتنبرگ.

فصل ۳ – داشبورد و محتوا: پست، صفحه، رسانه و ادیتور بلوکی

🎯 هدف: هسته کار روزمره وردپرس: تولید محتوا. فرق Post و Page، دسته/برچسب، کتابخانه رسانه و ادیتور بلوکی گوتنبرگ – از نگاه کاربری و از نگاه دولوپری (چون همین محتواها بعداً از API می‌خوانیم).

۳.۱ – Posts در برابر Pages: فرق بنیادی

Page (برگه)	Post (نوشته)	
محتوای ثابت (درباره ما، تماس)	محتوای زمان‌دار (وبلاگ، خبر، مقاله)	ماهیت
اهمیتی ندارد	دارد ★	تاریخ انتشار
ندارد	دارد	دسته/برچسب
نمی‌آید	می‌آید	در فید/آرشیو
«منوی رستوران»	«تخفیف پاییزه اومد!»	مثال

در API هم جدا میان: `/wp/v2/posts` و `/wp/v2/pages` – وقتی به فصل ۹ بررسی این تقسیم رو عیناً می‌بینی.

۳.۲ – اولین پست: گردش کامل

Posts → Add New و آشنایی با ادیتور بلوکی (Gutenberg):

- عنوان بالا، متن پایین – ولی متن از **بلوک‌ها** ساخته می‌شه
- Enter = بلوک جدید؛ / = منوی جستجوی بلوک (مثلاً `/code`، `/table`، `/image`)
- بلوک انتخاب‌شده = تنظیماتش در نوار راست (Sidebar) – رنگ، اندازه، align

پرکاربردترین بلوک‌ها:

بلوک	کار	معادل در ذهن تو
Paragraph / Heading	متن و تیتر	<code><p></code> / <code><h2></code>
Image / Gallery	تصویر / چندتایی	<code></code> / grid
List / Table / Quote	لیست / جدول / نقل قول	<code></code> / <code><table></code> / <code><blockquote></code>
Code / Embed	کد / ویدیو یوتیوب	<code><pre></code> / <code><iframe></code>
Columns	چند ستونه	flex/grid

دسته‌بندی و برچسب (پنل راست پایین): - **Categories** = ساختار درختی اصلی (مثلاً «آموزش» < «وردپرس») - یکی اجباری - **Tags** = برچسب‌های عرضی آزاد («تخفیف»، «جدید»)

Featured Image (تصویر شاخص): عکس اصلی پست - در API به صورت `_links` و با GraphQL عیناً میاد. برای لیست‌های Next.js حیاتیه؛ همیشه بذار.

بعد **Publish** - و در سایت (Open Site) ببینش.

۳.۳ - صفحات و اولین صفحه ثابت

یک Page بساز: «درباره ما» + چند بلوک. بعد یک صفحه «خانه» (Home) با چند بلوک معرفی.

حالا مهم‌ترین تنظیم این فصل:

Settings → Reading → Your homepage displays: بذار روی «Your latest posts» بمون (پیش‌فرض) - در Headless فرانت خودت صفحه اصلی رو می‌سازی، پس وردپرس فقط داده‌ها رو می‌ده.

۳.۴ - کتابخانه رسانه (Media Library)

Media → Add New: همه تصاویر/فایل‌ها اینجا جمع می‌شن، با:

- **Alt Text** ★ (متن جایگزین) - سئو و دسترس‌پذیری؛ همیشه پر کن (در API میاد: `alt_text`)
- Title/Caption/Description
- در Headless: URL عکس‌ها رو از API می‌گیری و در `next/image` می‌ذاری (فصل ۱۲) - پس اینجا فقط کیفیت و `alt` مهمه

⚠ نکته چاقی سایت: وردپرس از هر عکس چند سایز می‌سازه. عکس‌های سنگین (۵ مگ +) آپلود نکن؛ قبلش فشرده کن.

۳.۵ – آنچه پشت صحنه ذخیره شد (نگاه مهندسی)

نگاه سریع به دیتابیس (phpMyAdmin از Adminer/Studio → Database tab → Local):

- جدول `wp_posts`: همه پست‌ها، صفحات و حتی revision ها (با ستون `post_type`)
- جدول `wp_term_relationships` + `wp_terms`: دسته‌ها و برچسب‌ها
- جدول `wp_users`: کاربران

🔑 «همه‌چیز post است» – فلسفه وردپرس. حتی پیوست‌ها (attachment) و صفحه‌های ثابت، `post_type` های متفاوت از یک جدول‌ان. در فصل ۷ خودت `post_type` جدید می‌سازی (مثلاً «محصول») و همین ساختار برایش ساخته می‌شه.

✓ جمع‌بندی فصل

- Post = محتوای زمان‌دار با دسته/برچسب؛ Page = ثابت – در API جدا هستند
- ادیتور بلوکی: / = جستجوی بلوک؛ بلوک = کامپوننت؛ تنظیمات در Sidebar
- Featured Image همیشه بگذار – سوخت لیست‌های Next.js
- Media: Alt Text اجباری؛ عکس سنگین آپلود نکن
- زیر hood: همه‌چیز در `wp_posts` با `post_type`

📝 تمرین فصل ۳

۱. ۵ پست وبلاگی (با موضوع دلخواه: مثلاً معرفی ابزارهای دولوپری) با ۲-۳ دسته، برچسب، عکس شاخص و حداقل ۴ نوع بلوک مختلف بنویس – این داده‌ها سوخت پروژه Headless فصل ۱۱ است، با کیفیت بنویس!
۲. دو صفحه «درباره ما» و «تماس با ما» بساز.
۳. در phpMyAdmin جدول `wp_posts` را باز کن و ستون `post_type` را ببین – چه مقادیری دیدی؟ (`page` , `post` , `attachment` , `revision` ...)
۴. (کشف) در ادیتور بلوکی، :: کنار هر بلوک را بزن – می‌تونی بلوک را جابجا/تکرار کنی. و `%/Ctrl+Z` هم کار می‌کند 😊

▼ جواب تمرین ۳

مقادیر رایج: `post` (نوشته‌ها)، `page` (برگه‌ها)، `attachment` (رسانه‌ها)، `revision` (نسخه‌های خودکار)، `nav_menu_item` (آیتم‌های منو) – و هر CPT که بسازی (فصل ۷) اینجا ستون می‌شود.

→ فصل بعد: تم‌ها – ظاهر سایت، Site Editor و مفهوم تم فرزندان.

فصل ۴ – تم‌ها: ظاهر سایت، Site Editor و تم فرزند

🎯 هدف: اکوسیستم تم‌ها رو بشناسی، یک تم نصب و شخصی‌سازی کنی، و مهم‌ترین مفهوم توسعه‌ای: تم فرزند (Child Theme). برای Headless تم کمتر مهمه – ولی فهم تم = فهم قلب وردپرسه، و در پروژه‌های Classic هم نیازت می‌شه.

۴.۱ – تم: قالب نمایش محتوا

همون محتوا (فصل ۳) با تم‌های مختلف، ظاهرهای کاملاً متفاوت می‌گیره. دو نسل تم:

نسل	چیه	شخصی‌سازی
Classic Themes	تم‌های PHP سنتی (...Twenty Twenty-One)	Customizer (پنل قدیمی)
Block Themes ★	تم‌های بلوکی مدرن – کل قالب از بلوک‌ها ساخته شده	Site Editor (ادیتور همان گوتنبرگ برای کل سایت!)

وردپرس 6.9 به سمت Block Themes رفته و تم پیش‌فرض‌های جدید (Twenty Twenty-Five/Six) همگی Block Theme اند.

۴.۲ – نصب و تست تم

Themes → Add New → Appearance → جستجو/نصب رایگان از مخزن وردپرس → **Activate**.

تم‌های خوب رایگان برای شروع: **Twenty Twenty-Five** (پیش‌فرض)، **Astra**، **GeneratePress** (دوومی سبک و محبوب برای Classic).

بعد از Activate، سایت رو رفرش کن – همین! ظاهر عوض شد، محتوا همونه. (همون جداسازی data/view که تو در React با props/children می‌بینی.)

۴.۳ – Site Editor: طراحی با بلوک‌ها

Appearance → Editor (در Block Themes): کل سایت قابل ویرایش با بلوک‌ها:

- **Templates**: قالب‌های صفحه (صفحه اصلی، تک‌پست، آرشیو...) – ساختار صفحه
- **Styles**: رنگ‌ها و تایپوگرافی سراسری (مثل design tokens!)

• **Navigation / Header / Footer**: به صورت بخش‌های template part

تمرین: در Styles یک رنگ اصلی و فونت عوض کن → Save → سایت رو ببین.

💡 برای تو: Visual Studio Code = Site Editor برای HTML/CSS به زبان بلوکی. و Templates دقیقاً همون layout های Next.js ان (`app/(shop)/layout.tsx` فکر کن) – فقط با بلوک.

۴.۴ Customizer (برای تم‌های کلاسیک)

Appearance → Customize: پنل شخصی‌سازی تم‌های قدیمی (لوگو، رنگ‌ها، منوها). اگر تم Classic نصب کردی اینو می‌بینی؛ برای Block Themes منسوخه. فقط بدون وجود داره.

۴.۵ منوها (Navigation)

Appearance → Menus (یا در Site Editor، بلوک Navigation): منوی بالای سایت رو با صفحات/دسته‌ها بساز. در Headless از API خونده می‌شه (`wp/v2/menu-items` یا GraphQL) – پس اینجا منو رو درست بساز که داده‌اش تمیز باشه.

۴.۶ Child Theme: قانون طلایی توسعه

هیچ‌وقت فایل‌های تم اصلی رو مستقیم ویرایش نکن – با اولین آپدیت تم، همه تغییرات می‌پره!
راه‌حل: تم فرزند – تمی که «از تم مادر ارث می‌بره» و فقط چیزهای تغییر یافته رو override می‌کنه:

```
wp-content/themes/
├── twentytwentyfive/
└── twentytwentyfive-child/
    ├── style.css
    └── functions.php
```

تم مادر (دست نمی‌زنیم – آپدیت می‌شود) ←
تم فرزند (کد تو) ←
هدر اجباری + استایل‌ها ←
تو PHP کدهای ←

تم فرزند فقط یک هدر کامنت لازم داره: `style.css`

```
/*
Theme Name: Twenty Twenty-Five Child
Template: twentytwentyfive ← ⭐ رابطه فرزندگی
*/
```

فعلاً فقط مفهومش رو بگیر: **تغییرات تو همیشه در لایه‌ای جدا از هسته/تم اصلی** – همون فلسفه‌ای که در کد هم داری (قبل از override کردن library، fork کن!).

🔦 ساخت سریع با WP-CLI (فصل ۲): `wp scaffold child-theme twentytwentyfive-child --parent=twentytwentyfive`

۴.۷ – تم‌های premium و Page Builder ها (آگاهی)

در بازار ایران: تم‌های premium (مثل Enfold، Flatsome) و **Page Builder** ها (Elementor، WPBakery) – ابزارهای درگ‌اند‌دراپ صفحه‌سازی. بدون که: Elementor محبوب‌ترین و بسیاری از پروژه‌های ایرانی باهاش ساخته می‌شن - ولی برای Headless به درد نمی‌خورن (خروجی‌شون HTML تمپلیت است نه داده ساخت‌یافته) – Headless مسیر خودش رو داره: CPT + ACF + API (فصل‌های ۷ به بعد)

✅ جمع‌بندی فصل

- دو نسل تم: Classic (Customizer) و **Block (Site Editor)** – آینده با Block
- Site Editor = ویرایش کل سایت با بلوک (Templates + Styles)
- منوها را تمیز بساز – در Headless از API می‌آیند
- **تم فرزند = لایه تغییرات جدا از تم اصلی** – هرگز فایل تم اصلی را ویرایش نکن
- Page Builder ها (Elementor و...) برای Headless به درد نمی‌خورند

📝 تمرین فصل ۴

1. دو تم نصب و فعال کن و فرق ظاهر را ببین؛ بعد Twenty Twenty-Five را برگردان.
2. در Site Editor: رنگ اصلی سایت و فونت را عوض کن و Save – سایت را ببین.
3. منوی بالای سایت را با صفحات فصل ۳ (درباره ما، تماس) و یک دسته‌بندی بساز.
4. با WP-CLI یک تم فرزند بساز و فعالش کن (`wp theme activate ...` + `wp scaffold child-theme ...`) – سایت همچنان کار می‌کند؟
5. (کشف) در Site Editor یک Template را باز کن (مثلاً Single) و ساختار بلوکی‌اش را ببین – این همان «layout» است.

```
# در Open Site Shell:  
wp scaffold child-theme tt5-child --parent=twentytwentyfive  
wp theme activate tt5-child  
wp theme list --status=active
```

اگر پوشه تم مادر اسم دیگری دارد، `--parent` را با همان نام پوشه بده.

➔ **فصل بعد:** پلاگین‌ها – قدرت‌های اضافه: SEO، امنیت، فرم و بیشتر.

فصل ۵ – پلاگین‌ها: قابلیت‌های اضافه و ضروری‌ها

🎯 هدف: اکوسیستم پلاگین (npm دنیای وردپرس) – چطور نصب/غیرفعال/حذف کنی، چه پلاگین‌هایی برای هر پروژه ضروریه، و زیر hood یک پلاگین چیه. آخر فصل، عادت‌های امنیتی نصب رو هم داری.

۵.۱ – پلاگین = npm package وردپرس

هر قابلیت که در هسته نیست: فروشگاه (WooCommerce)، فرم تماس، SEO، مولتی‌زبان، کش... همه پلاگین‌اند:

وردپرس	npm	
premium + wordpress.org/plugins بازارهای	npmjs.com	مخزن
Plugins → Add New → Install	<code>npm install x</code>	نصب
دکمه Activate ⚠️	import در کد	فعال‌سازی
Deactivate → Delete	<code>npm uninstall</code>	حذف

⚠️ تفاوت مهم: پلاگین نصب‌شده ≠ فعال. Deactivated فقط خاموشه (داده‌اش می‌مونه)؛ Delete واقعاً پاکش می‌کنه.

سه راه نصب: 1. **Add New** از مخزن رسمی (رایگان) – استاندارد 2. **Upload Plugin** (فایل zip پولی/اختصاصی) 3. WP-CLI: `wp plugin install woocommerce --activate` ⭐ (سریع‌ترین – از دوره CLI بلدی!)

۵.۲ – پلاگین‌های ضروری هر پروژه (چک‌لیست)

دسته	پلاگین پیشنهادی	کار
SEO	Yoast SEO یا Rank Math	متا تگ‌ها، sitemap، اسکیمای – (فصل ۱۲: خروجیش در Next.js مصرف می‌شه)
امنیت	Wordfence یا Solid Security	فایروال، اسکن، محدود کردن لاگین
بکاپ	UpdraftPlus	بکاپ خودکار زمان‌بندی‌شده (فایل + دیتابیس)
کش / سرعت	LiteSpeed Cache یا W3 Total Cache	کش صفحات (در Headless نقشش کمتر می‌شه)
فرم	Contact Form 7 یا Fluent Forms	فرم تماس (در Headless: فصل ۱۲ راه مدرن)
اسپم	Akismet (پیش‌فرض نصبه)	ضد اسپم کامنت

💡 قانون طلایی پلاگین: کمتر = بهتر. هر پلاگین کد PHP اجرا می‌کنه – پلاگین زیاد یعنی کندی و سطح حمله بیشتر. فقط ضروری‌ها. (مثل بوت‌استرپ + ۲۰ تا UI library روی هم، ولی سنگین‌تر!)

۵.۳ – نصب اولین پلاگین: Yoast SEO

1. Plugins → Add New → جستجوی **Activate** → Install → Yoast SEO
2. منوی جدید Yoast SEO ظاهر می‌شه – Wizard اولیه رو رد کن (پیش‌فرض‌ها اوکیه)
3. حالا در صفحه ویرایش هر پست، یک باکس Yoast پایین اضافه شده: عنوان و توضیحات متا + پیش‌نمایش گوگل
4. در فصل ۱۲ این مقادیر متا رو از API می‌خونیم و در Next.js `metadata` می‌ذاریم – پس همین الان برای پست‌های فصل ۳، عنوان سئو بذار!

۵.۴ – زیر hood یک پلاگین (نگاه مهندسی)

هر پلاگین = یک پوشه در `wp-content/plugins/` با حداقل یک فایل PHP که هدر مخصوص داره:

```
<?php
/**
 * Plugin Name: My First Plugin
 * Description: سلام از طرف من!
 * Version: 1.0.0
 * Author: Mahdi
 */
```

همین! فایل PHP با این هدر = پلاگین قابل فعال‌سازی. (در فصل ۸ اولین پلاگین واقعی خودت رو می‌نویسی.)

ببین پلاگین‌های نصب‌شده رو: Local → پوشه سایت → `wp-content/plugins/` – هر پوشه، یک پلاگین.

۵.۵ – عادات‌های امنیتی و نگهداری

- پلاگین را فقط از مخزن رسمی یا منبع معتبر نصب کن (پلاگین نال‌شده = اسب تروا!)
- آپدیت‌ها را جدی بگیر – اکثر هک‌های وردپرسی از پلاگین‌های قدیمی (Plugins → Updates)
- پلاگین غیرفعال بلااستفاده = حذفش کن
- قبل از هر تغییر بزرگ: Snapshot در Local (فصل ۲)
- تعداد پلاگین فعال را زیر ۱۰ نگه دار

و WP-CLI برای نگهداری سریع:

```
wp plugin list           # وضعیت همه
wp plugin update --all   # آپدیت همه
wp plugin deactivate akismet # غیرفعال
wp plugin delete akismet  # حذف کامل
```

جمع‌بندی فصل

- پلاگین = قابلیت آماده؛ `!Install ≠ Activate`؛ `wp plugin install x --activate`
- ضروری‌ها: (Yoast) SEO، امنیت، بکاپ، کش – و قانون کمتر = بهتر
- زیر hood: پوشه PHP با هدر `Plugin Name`
- امنیت: منبع معتبر، آپدیت منظم، حذف بی‌استفاده‌ها، Snapshot قبل از تغییر

تمرین فصل ۵

1. Yoast SEO را نصب و فعال کن و برای سه پست فصل ۳، عنوان و توضیح سئو بنویس.
2. UpdraftPlus را نصب و یک بکاپ دستی بگیر – فایل‌هاش را پیدا کن.
3. با `wp plugin list` WP-CLI: بزن؛ بعد با CLI یک پلاگین نصب و فعال کن (مثلاً `contact-form-7 --activate`) و در پنل ببینش.
4. پلاگینی را غیرفعال کن و سایت را ببین (چه چیزی غیب شد؟)؛ بعد بکش up.
5. (کشف) پوشه `wp-content/plugins/` را در فایل اکسپلورر باز کن – چند پلاگین چند فایل رو ببین (ووکامرس را حتماً ببین اگر نصب کردی – ۳۰۰+ فایل!).

▼ نکته تمرین ۲

UpdraftPlus → Settings → Backup Now. فایل‌ها در `wp-content/updraft/` ذخیره می‌شوند (و می‌توانی به گوگل‌درایو وصلش کنی). برای پروژه‌های واقعی: بکاپ خودکار هفتگی = بیمه شغلی.

→ **فصل بعد:** کاربران، نقش‌ها و تنظیمات کلیدی – از جمله Permalink که پایه API است.

فصل ۶ – کاربران، نقش‌ها و تنظیمات کلیدی

🎯 هدف: مدیریت کاربران و نقش‌های دسترسی + تنظیماتی که برای پروژه Headless حیاتی‌ان: **Permalinks** (ساختار URL – مستقیم روی API اثر داره!) و تنظیمات خواندن. آخر فصل، یک User Editor برای مشتری هم ساختی.

۶.۱ – نقش‌های کاربری: چه کسی چه اجازه‌ای داره؟

وردپرس ۵ نقش پیش‌فرض داره (از کم‌اختیار به پراختیار):

نقش	کارش	در پروژه واقعی کیه؟
Subscriber	فقط پروفایل خودش (کامنت با اسم خودش)	کاربر عادی سایت
Contributor	می‌نویسه، ولی انتشار دست دیگری است	نویسنده تازه‌کار
Author	پست‌های خودش را منتشر می‌کند	نویسنده
Editor	همه محتوا: همه پست‌ها، صفحات، دسته‌ها، کامنت‌ها	سرمدبیر / مدیر محتوا ★
Administrator	همه‌چیز: پلاگین، تم، کاربران، تنظیمات	تو (در لوکال) / مدیر فنی

قاعده‌ی امنیتی مهم برای پروژه واقعی: مشتری را **Editor** بده، نه **Admin**. چون Admin می‌تونه پلاگین نصب کنه و سایت رو بترکونه. خودت (فنی) Admin می‌مونی.

Users → Add New: یک کاربر `writer` با نقش Author بساز و یک بار باهاش لاگین کن (مرورگر ناشناس) – پنلش رو ببین: پلاگین و تنظیمات غیب شدن!

۶.۲ – Permalinks: ساختار URL ها ★ (برای Headless حیاتی)

Settings → Permalinks – این صفحه تعیین می‌کنه URL پست‌ها چطوری باشه:

گزینه	نمونه URL	نظر
Plain	?p=123	✗ سئوی صفر
Day and name	/عنوان/2026/09/13/	شلوغ
★ Post name	/عنوان-پست/	استاندارد سئو – انتخاب کن!
Custom	دلخواه	پیشرفته

چرا برای تو حیاتی است؟

1. Slug (قسمت آخر URL) = شناسه‌ای که در Next.js برای routing استفاده می‌کند: slug ← `/blog/[slug]` از API میاد ← اینجا تعیین می‌شه که slug چی باشه

2. با تغییر ساختار URL، permalink، های قدیمی می‌شکنن – پس **اول پروژه تعیینش کن، بعد دست نزن**

در ویرایشگر هر پست هم می‌تونی slug رو دستی تمیز کنی (عنوان فارسی → slug خودکار فارسی؛ برای URL تمیزتر می‌تونی لاتین بزنی: `best-tools-2026`).

۶.۳ – تنظیمات خواندن و دیدن (Reading)

Settings → Reading:

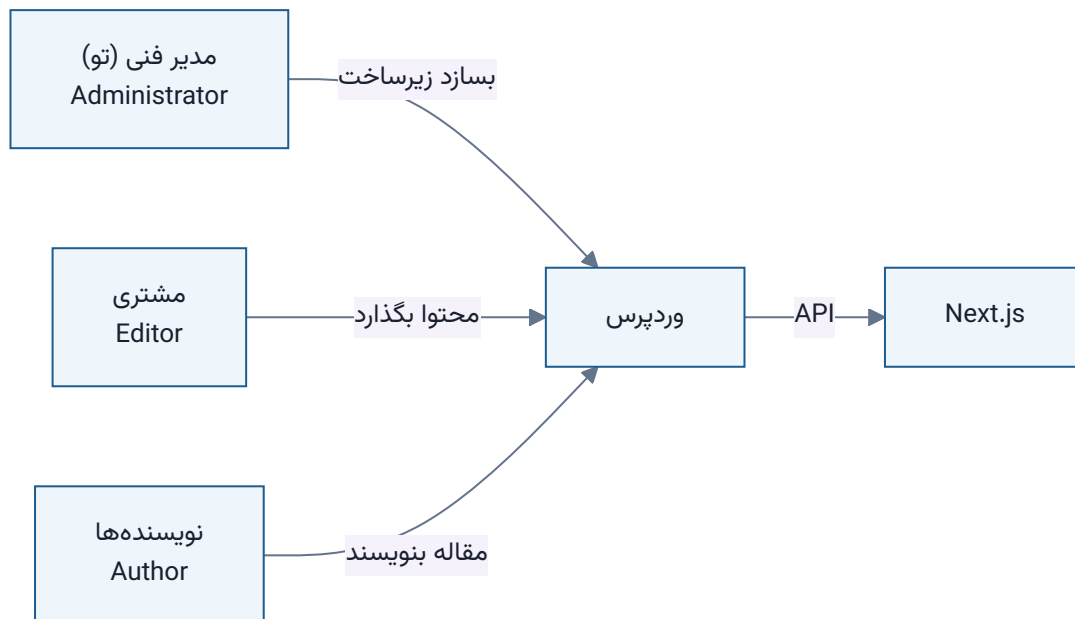
- **Your homepage displays:** «Your latest posts» (پیش‌فرض) یا یک Page ثابت به عنوان خانه
- **Blog pages show at most:** تعداد پست در هر صفحه آرشیو
- **Discourage search engines** ☒ **Search engine visibility:** ☒ – این تیک را هرگز در سایت واقعی نزن (سایت از گوگل حذف می‌شود!) فقط برای سایت‌های موقت تست خوبه

۶.۴ – تنظیمات مهم دیگر (گشت سریع)

مسیر	تنظیم مهم	نکته
General	Site Title / Tagline	عنوان و شعار سایت (در <code>name</code> , <code>description</code> API)
General	Timezone	Tehran
Discussion	کامنت‌ها	مدیریت نظرها – در Headless معمولاً کامنت را از سیستم دیگری می‌گیریم (فصل ۱۲)
Media	سایزهای تصویر	اندازه‌های خودکار تولیدی (فصل ۱۲: در <code>next/image</code> مفید)
Privacy	صفحه حریم خصوصی	صفحه policy بساز (برای فرم‌ها لازم می‌شه)

۶.۵ – گردش کار تیمی نمونه (پروژه واقعی)

سناریوی «سایت فروشگاهی با تیم ۳ نفره»:



تو زیرساخت رو می‌سازی (فصل ۷: CPT و فیلدها)، مشتری فقط محتوا می‌ریزه، و Next.js نشون می‌ده – مسئولیت‌ها تمیز جدا شدن.

✓ جمع‌بندی فصل

- نقش‌ها: Editor < Administrator < Author < Contributor < Subscriber – مشتری = Editor
- **Permalink روی «Post name»** – و slug تمیز برای routing در Next.js
- Reading: صفحه اصلی پیش‌فرض؛ تیک «Discourage search engines» خطرا!
- General/Timezone/Discussion/Media – سریع تنظیم کن

📝 تمرین فصل ۶

1. Permalink را روی «Post name» بذار؛ یک پست با slug دستی لاتین بساز (`my-first-post`) و URL اش را ببین.
2. کاربر `writer` با نقش Author بساز و در مرورگر ناشناس باهاش لاگین شو – چه منوهایی را نمی‌بینی؟
3. کاربر `editor-boss` با نقش Editor بساز – چه اختیاراتی نسبت به Author بیشتره؟ (بازش کن و مقایسه کن)
4. تنظیمات General: عنوان سایت را اسم پروژه‌ات کن و Timezone را Tehran.
5. (برای فصل ۱۲) صفحه «حریم خصوصی» را از Settings → Privacy بساز.

▼ نکته تمرین ۳

Editor نسبت به Author: می‌تواند پست‌های **دیگران** را ویرایش/انتشار کند، صفحات و دسته‌ها را مدیریت کند و کامنت‌ها را مدیریت کند – ولی پلاگین/تم/کاربر (اختیارات Admin) ندارد.

➔ **فصل بعد:** مهم‌ترین فصل نیمه اول – Custom Post Types و Custom Fields: محتوای ساخت‌یافته‌ای که Headless با آن ساخته می‌شود!

فصل ۷ – Custom Post Types و Custom Fields: محتوای

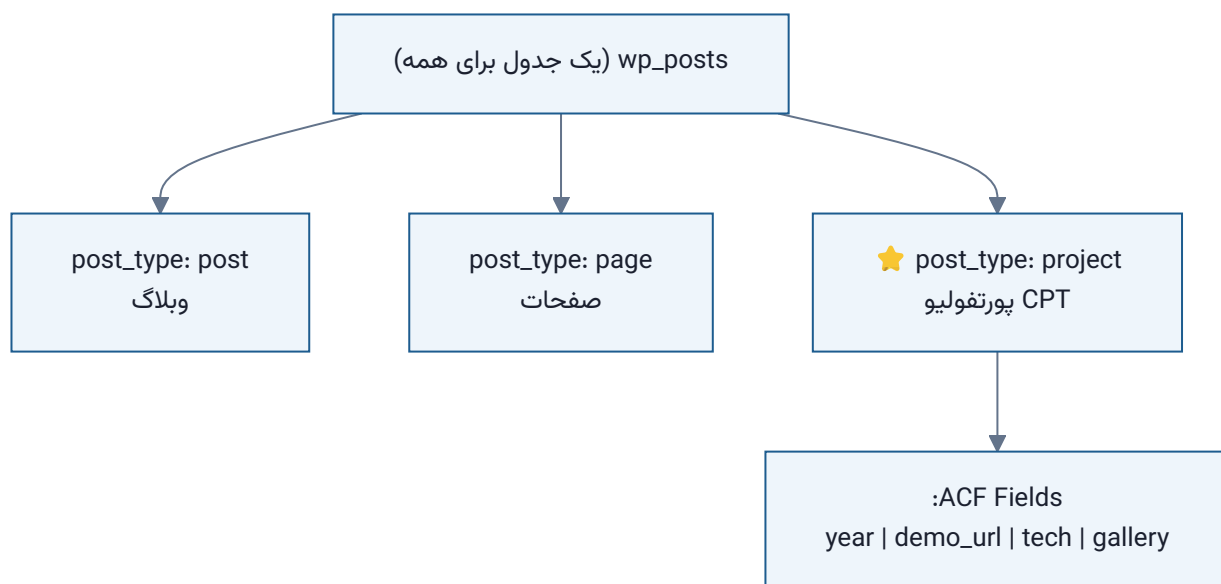
ساخت یافته ★

🎯 مهم‌ترین فصل نیمه اول! پست و صفحه برای «وبلاگ» کافیه، ولی پروژه واقعی «موجودیت» داره: محصول، پروژه پورتفولیو، نمای ملک، دوره آموزشی... یاد می‌گیری CPT بسازی و با ACF فیلدهای ساخت یافته بهش بدی – این دقیقاً همون مدل داده‌ای است که در Next.js به صورت typed مصرف می‌کنی.

۷.۱ – مسئله: پست برای «محصول» مناسب نیست

سناریو: سایت پورتفولیو می‌سازی؛ مشتری می‌خواه پروژه اضافه کنه با: عنوان، توضیح، عکس، سال انجام، لینک دمو، تکنولوژی‌ها. با پست معمولی؟ همه‌چیز می‌ریزه توی یک textarea آزاد – بدون ساختار، بدون کنترل.

جواب وردپرس: هر نوع محتوا = یک Custom Post Type (CPT) – مثل اینکه `post_type` جدید تعریف کنی. (یادت هست فصل ۳: همه‌چیز در جدول `wp_posts` با ستون `post_type` است؟ حالا داریم به همین ستون، نوع خودمون رو اضافه می‌کنیم!)



۷.۲ – ساخت CPT: دو راه

راه ۱: کد (روش حرفه‌ای ★)

در `functions.php` تم فرزند (فصل ۱۴) یا در یک پلاگین اختصاصی (فصل ۸):

```

add_action('init', function () {
    register_post_type('project', [
        'labels' => [
            'name' => 'پروژه‌ها',
            'singular_name' => 'پروژه',
        ],
        'public' => true, // نمایش داده شود API در سایت و
        'has_archive' => true, // صفحه آرشیو داشته باشد
        'menu_icon' => 'dashicons-portfolio',
        'supports' => ['title', 'editor', 'thumbnail', 'excerpt'],
        'show_in_rest' => true, // ★★ در REST API بدون - ظاهر شود
        'rewrite' => ['slug' => 'projects'],
    ]);
});

```

بعد از ذخیره: در پنل، منوی جدید «پروژه‌ها» ظاهر شد! ★ (اگر نیامد: Settings → Permalinks → Save یک بار بزن – قانون رایج رفرش قواعد URL).

راه ۲: پلاگین UI (سریع‌تر برای شروع)

پلاگین Add New CPT (CPT UI) → Custom Post Type UI → همین فیلدها را با فرم پر کن. خروجی‌اش همان کد بالا است.

💡 نکته دولوپری: برای پروژه CPT، Headless را با کد بساز (در پلاگین خودت) تا version-able باشد – مثل تفاوت تنظیم دستی و Infrastructure as Code.

۷.۳ – Custom Fields با ACF: ساختار دقیق

CPT ساختار نوع محتوا را می‌دهد؛ فیلدهای دقیق (سال، لینک، تکنولوژی‌ها) را Custom Fields می‌دهند. پادشاه این حوزه پلاگین ACF (Advanced Custom Fields) است:

1. نصب (ACF (Plugins → Add New → Advanced Custom Fields → Activate)
2. Add New → Field Groups → ACF: اسم: «اطلاعات پروژه»
3. Location rule: «Post Type is equal to project» – یعنی این فیلدها فقط روی CPT پروژه ظاهر شوند
4. فیلدها را اضافه کن:

Field Label	Field Name (نام)	نوع	نکته
سال انجام	year	Number	
لینک دمو	demo_url	URL	
تکنولوژی‌ها	technologies	Checkbox (React, Next, Node...)	چند انتخابی
گالری	gallery	Gallery	چند عکس

5. حالا پروژه‌ها → Add New: فرم اختصاصی توست! چند پروژه با داده‌های واقعی وارد کن (۳-۴ تا با عکس – سوخت پروژه Next.js!)

🔑 Field Name (نام انگلیسی) همان کلیدی است که در API می‌گیری – مثل prop های کامپوننت. تمیز و کوتاه
 بز (year نه «سال انجام پروژه»).

۷.۴ – خروجی API را همین حالا ببین! (پیش‌تaste فصل ۹)

در مرورگر: `wp-course.local/wp-json/wp/v2/projects` – جیسون پروژه‌هایت را می‌بینی! و فیلدهای ACF با پلاگین **ACF to REST API** هم اضافه می‌شوند (`acf: {...}` در خروجی).

همین یک URL یعنی: وردپرس الان یک بک‌اند استاندارد JSON برای محتوای توست – و در فصل‌های بعد از داخل Next.js می‌خوانیمش.

۷.۵ – Taxonomy اختصاصی (اختیاری ولی ارزشمند)

برای پروژه‌ها می‌خواهی «دسته‌بندی مهارت» هم داشته باشی؟ Custom Taxonomy:

```
add_action('init', function () {
  register_taxonomy('tech_stack', 'project', [
    'labels' => ['name' => 'تکنولوژی‌ها', 'singular_name' => 'تکنولوژی'],
    'public' => true,
    'show_in_rest' => true,           // ★ باز هم برای API
    'hierarchical' => false,         // مثل تگ، نه دسته درختی
  ]);
});
```

حالا پروژه‌ها را می‌توانی با فیلتر تکنولوژی آرشیو کنی – در `Next.js: /projects?tech=react`.

جمع‌بندی فصل ✓

- محتوای ساخت‌یافته = CPT (نوع محتوا) + ACF (فیلدهای دقیق)

- CPT با `register_post_type` یا پلاگین CPT UI؛ حتماً `show_in_rest => true`
- ACF: Field Group + Location rule؛ `Field Name` = کلید API
- پروژه‌های نمونه را همین حالا با داده واقعی بگذار – سوخت فصل ۱۱
- Taxonomy اختصاصی = فیلتر/آرشیو ساخت‌یافته

تمرین فصل ۷ (پروژه‌محور – تا آخر دوره باهش کار می‌کنیم!)

1. `project` CPT را با کد (در تم فرزند) بساز – با `show_in_rest` و slug اختصاصی.
2. ACF را نصب کن و Field Group «اطلاعات پروژه» را دقیقاً مثل جدول بالا بساز.
3. سه پروژه واقعی (از رزومه‌ات!) با عکس شاخص و فیلدهای کامل وارد کن.
4. `wp-json/wp/v2/projects` را در مرورگر باز کن – خروجی JSON پروژه‌هایت را ببین (عنوان، slug، آی‌دی).
5. پلاگین **ACF to REST API** را نصب و فعال کن و همان URL را رفرش کن – بخش `acf` را در JSON پیدا کن
(`year` , `demo_url` , `technologies`).
6. `tech_stack` Taxonomy را اضافه کن و به پروژه‌ها تکنولوژی بچسبان.

چک‌لیست تمرین ۴-۵ ▼

- `/wp-json/wp/v2/projects` → آرایه‌ای از آبجکت‌ها: `id` , `slug` , `title.rendered` , `_links`
- بعد از ACF to REST API: هر آبجکت یک کلید `acf` دارد →
`{"year": "2026", "demo_url": "...", "technologies": ["React", "Next"]}`
- اگر 404 گرفتی: Permalink را دوباره Save کن و `show_in_rest` را چک کن – دو پرتکرارترین گیرهای این مرحله!

➔ **فصل بعد:** کمی PHP برای دولوپر JS – تا کد وردپرسی‌ها را بخوانی و اولین پلاگینت را بنویسی.

فصل ۸ – PHP برای دولوپر JS: خواندن کد وردپرس و اولین پلاگین

🎯 هدف: قرار نیست PHP کار بشی – قراره به اندازه‌ای PHP بلد بشی که (۱) کد تم‌ها و پلاگین‌ها رو بخونی، (۲) فهم دقیق Hooks داشته باشی، و (۳) اولین پلاگین کاربردی خودت رو بنویسی. با ذهنیت JS/TS که داری، سریع‌تر از فکر می‌گیری.

۸.۱ – PHP در ۵ دقیقه (با مقایسه به JS)

مفهوم	JS	PHP
متغیر	<code>let x = 1;</code>	<code>\$x = 1;</code> ← همیشه با <code>\$</code>
رشته	<code>'a' + b</code>	<code>"a" . \$b</code> ← نقطه = اتصال
آرایه	<code>[1, 2]</code>	<code>[1, 2]</code>
آبجکت/دیکشنری	<code>{a: 1}</code>	<code>['a' => 1]</code> ← <code>=></code>
تابع	<code>function f(\$a) {}</code>	<code>function f(\$a) {}</code> (آرگومان با <code>\$</code>)
خروجی	<code>console.log</code>	<code>echo / print_r</code>
شرط/حلقه	<code>if/for/foreach</code>	یکسان!
interpolate	<code>`hi \${n}`</code>	<code>"hi \$n"</code> (با دابل‌کوتهیشن)

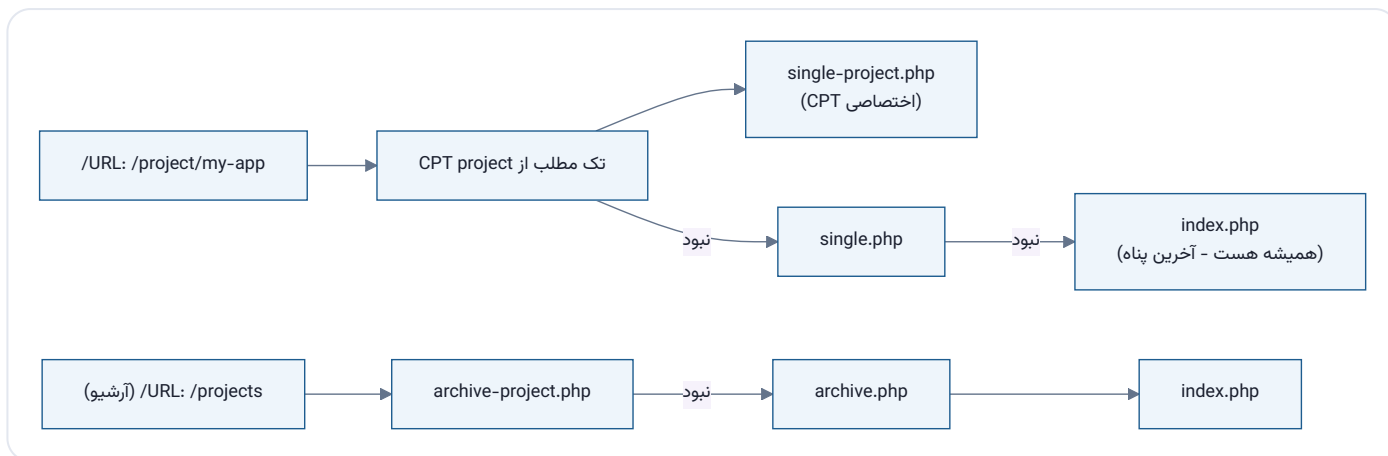
```
$projects = ['shop', 'blog'];
foreach ($projects as $p) {
    echo "پروژه: $p <br>";
}

$user = ['name' => 'Ali', 'role' => 'editor'];
echo $user['name']; // Ali
```

باقی‌اش با context یاد گرفته می‌شه – بذار بریم سراغ جاهایی که واقعاً لازمش داری.

۸.۲ – Template Hierarchy: وردپرس کدام فایل را نشان می‌دهد؟

وقتی بازدیدکننده یک URL را باز می‌کند، وردپرس با یک درخت تصمیم مشخص می‌کند از کدام فایل تم استفاده کند:



قاعده: از اختصاصی به عمومی. `single-{posttype}.php` برای تک‌نمایش هر `archive-{posttype}.php`، برای CPT، آرشیوش. (در Block Themes این‌ها template های Site Editor هستند – مفهوم یکی است).

۸.۳ – The Loop: قلب نمایش وردپرس

الگوی کلاسیک تم PHP برای پیمایش پست‌ها:

```

<?php if (have_posts()) : ?>
  <?php while (have_posts()) : the_post(); ?>
    <h2><a href="<?php the_permalink(); ?>"><?php the_title(); ?></a></h2>
    <div><?php the_excerpt(); ?></div>
  <?php endwhile; ?>
<?php else : ?>
  <p>چیزی یافت نشد</p>
<?php endif; ?>

```

به زبان تو:

```

if (posts.length) {
  for (const post of posts) {
    console.log(post.title, post.permalink);
  }
} else {
  console.log("چیزی یافت نشد");
}

```

دقیقاً یک `map` روی داده‌هاست – فقط با توابع کمکی وردپرس (`the_title()` و ...). تا اینجا PHP رو که فهمیدی، ۸۰٪ تم‌ها را می‌خوانی.

۸.۴ – Hooks: سیستم رویداد وردپرس ★ (مهم‌ترین مفهوم این فصل)

وردپرس در مسیر اجرایش هزاران نقطه قلاب دارد که تو می‌توانی کدت را آویزان کنی – بدون تغییر هسته. دو نوع:

نوع	کی اجرا می‌شود؟	می‌تواند چیزی را عوض کند؟	مثال
Action	«وقتی X اتفاق افتاد، این را هم بکن»	نه (فقط کار انجام می‌دهد)	event listener
Filter	«این مقدار را از من عبور بده»	بله – مقدار را تغییر می‌دهد و برمی‌گرداند	pipe / interceptor

```
// Action: بعد از فعال شدن پلاگین، چیزی انجام بده
add_action('init', function () {
    register_post_type('project', [ /* ... ۷ فصل ... */ ]);
});

// Filter: عنوان پست‌ها را هنگام نمایش تغییر بده
add_filter('the_title', function ($title, $id) {
    if (get_post_type($id) === 'project') {
        return $title . ' 🚀';
    }
    return $title; // همیشه مقدار را برگردان!
}, 10, 2); // اولویت + تعداد آرگومان‌ها
```

با همین دو تابع (`add_action` و `add_filter`) کل وردپرس قابل تغییره – فلسفه open/closed: هسته بسته، توسعه باز.

۸.۵ – اولین پلاگین واقعی خودت: «تولید پست از قالب»

فصل ۵ دیدی پلاگین = یک PHP با هدر. حالا یکی می‌سازیم که کاری مفید کند: موقع ساخت هر پروژه جدید، به‌طور خودکار مقدار فیلد `year` را سال جاری بگذارد:

در `wp-content/plugins/my-tools/my-tools.php`:

```
<?php
/**
 * Plugin Name: My Headless Tools
 * Description: ابزارهای پروژه Headless من
 * Version: 1.0.0
 */

// وقتی پروژه جدید ذخیره می‌شود، اگر سال خالی بود، سال جاری را بگذار
add_action('save_post_project', function ($post_id) {
    if (get_post_status($post_id) === 'publish') return;
    if (!empty(get_field('year', $post_id))) return; // get_field از ACF
    update_field('year', (int) date('Y'), $post_id); // update_field از ACF
});
```

پلاگین‌ها → Activate → یک پروژه جدید بدون سال بساز → publish → بازش کن: سال خودش پر شده! 🎉

تو الان یک پلاگین واقعی نوشتی که با `save_post_project` (event) کار می‌کند – دقیقاً همون ذهنیت `onClick` / `onChange` تو.

⚠ سه قانون توسعه پلاگین: (۱) هرگز هسته را ویرایش نکن – همه چیز با hook؛ (۲) همه چیز را در پلاگین/تم فرزند؛ (۳) قبل از تغییر مهم: Snapshot.

۸.۶ – ابزارهای دیباگ سریع

```
// console.log مثل) خروجی داده با فرمت خوانا
echo '<pre>'; print_r($data); exit;
// wp-content/debug.log یا به لاگ
error_log(print_r($data, true));
// wp-config.php فعال سازی دیباگ در
// define('WP_DEBUG', true); define('WP_DEBUG_LOG', true);
```

(exit مثل return زود هنگام اسکریپت را می کشد – برای دیدن یک متغیر وسط جریان.)

✓ جمع بندی فصل

- JS = PHP با \$ و . به جای +؛ آرایه انجمنی با ≥
- Template Hierarchy: از اختصاصی (single-project.php) به عمومی (index.php)
- The Loop با توابع کمکی = posts.map(...)
- Action = رویداد، Filter = تغییر مقدار – دو قلاب add_action / add_filter
- اولین پلاگین واقعی: hook روی save_post_project – بدون دست زدن به هسته

📝 تمرین فصل ۸

1. فایل single.php تم Twenty Twenty-Five را باز کن (یا پوشه تم را بگرد) و The Loop را در آن پیدا کن – به زبان JS اش بازنویسی کن (روی کاغذ).
2. پلاگین «My Headless Tools» را بساز، فعال کن و سناریوی سال خودکار را تست کن.
3. یک add_filter روی the_content بنویس که آخر همه پست های project جمله «ساخته شده با ❤» را اضافه کند – در سایت (نه API) تست کن.
4. (چالش) یک Action روی wp_insert_post بنویس که error_log کند عنوان هر پست جدید – و فایل debug.log را ببین (WP_DEBUG_LOG فعال کن).

▼ جواب تمرین ۳

```
add_filter('the_content', function ($content) {  
    if (get_post_type() === 'project' && is_singular('project')) {  
        return $content . '<p>ساخته شده با ❤️</p>';  
    }  
    return $content;  
});
```

توجه: `the_content` روی فرانت اعمال می‌شود؛ خروجی REST API پیش‌فرض فیلترهای خودش را دارد (در فصل ۹ می‌بینی).

🚀 **فصل بعد:** ورود به نیمه دوم – WP REST API: وردپرس به عنوان بک‌اند JSON. ➔

فصل ۹ – WP REST API: وردپرس به عنوان بک‌اند JSON

🎯 هدف: ورود رسمی به نیمه دوم! وردپرس از قبل یک API کامل REST دارد (`/wp-json`). یاد می‌گیری endpoint ها را با `curl` صدا بزنی، فیلتر کنی، و با `Application Password` احراز هویت کنی. این پایه‌ی فهم WPGraphQL هم می‌شود (فصل ۱۰).

۹.۱ – API رایگان که از قبل داری!

هیچ پلاگینی لازم نیست – وردپرس از ۴.۷ به بعد REST API کامل دارد. تستش کن (در مرورگر یا `curl`):

```
$ curl "http://wp-course.local/wp-json" | head -40
# ها endpoint نقشه کل API: ریشه

$ curl "http://wp-course.local/wp-json/wp/v2/posts"
# همه پست‌هایت! (همان‌هایی که فصل ۳ نوشتی) JSON آرایه

$ curl "http://wp-course.local/wp-json/wp/v2/projects"
# 🚧 پورتفولیوی فصل ۷ CPT
```

الگوی endpoint: `/wp-json/wp/v2/{نوع}`

نکته	چه چیزی	endpoint
فیلترهای قدرتمند می‌گیرد	پست‌ها	<code>/wp/v2/posts</code>
	صفحات	<code>/wp/v2/pages</code>
اگر <code>show_in_rest</code> بود (فصل ۷)	CPT تو!	<code>/wp/v2/projects</code>
	دسته/برچسب	<code>/wp/v2/categories / tags</code>
برای <code>next/image</code>	رسانه‌ها (عکس‌ها!)	<code>/wp/v2/media</code>
عمومی فقط عمومی‌ها	کاربران	<code>/wp/v2/users</code>
پایین‌تر	فیلتر	<code>/wp/v2/pages?slug=about</code>

۹.۲ – آناتومی پاسخ (چه چیزی می‌گیری؟)

بریده‌ای از خروجی یک پست:

```
{
  "id": 12,
  "date": "2026-09-13T10:00:00",
  "slug": "my-first-post",
  "status": "publish",
  "link": "http://wp-course.local/my-first-post/",
  "title": { "rendered": "عنوان پست" },
  "content": { "rendered": "<p>متن کامل با HTML...</p>" },
  "excerpt": { "rendered": "<p>خلاصه.</p>" },
  "featured_media": 34,
  "categories": [4],
  "_links": { "self": [...], "wp:featuredmedia": [...] }
}
```

نکته‌های دولوپری:

- ★ همان کلید routing تو در `/blog/[slug]` (Next.js) = `slug`
- `title.rendered` / `content.rendered` = **HTML** — در Next.js با `dangerouslySetInnerHTML` یا پارسر
- `featured_media` = فقط آی‌دی؛ عکس واقعی یا در `_links` است یا با یک درخواست دیگر (یا در GraphQL — اینجا است که GraphQL می‌درخشه!)
- مقادیر ACF با پلاگین «ACF to REST API» در کلید `acf` می‌آیند (فصل ۷ نصب کردی ✓)

۹.۳ – فیلترها: کوئری‌های واقعی

REST API وردپرس فیلترهای آماده دارد:

```
# خاص (برای صفحه جزئیات!) slug پست با:
$ curl "http://wp-course.local/wp-json/wp/v2/posts?slug=my-first-post"

# صفحه‌بندی + تعداد:
$ curl ".../wp/v2/posts?per_page=5&page=1"
# pagination! (برای کل تعداد = X-WP-Total نکته: هدر)

# فیلتر دسته، ترتیب، جستجو:
$ curl ".../wp/v2/projects?per_page=12&orderby=date&order=desc&search=react"
```

۹.۴ – احراز هویت: Application Passwords

خواندن محتوای منتشرشده عمومی؛ ولی **نوشتن/ویرایش** و دیدن **پیش‌نویس‌ها** نیاز به احراز هویت دارد. راه استاندارد وردپرس مدرن: **Application Password** (رمز جدا برای هر اپ، نه پسورد اصلی):

۱. پنل → Profile → Users → پایین صفحه: **Application Passwords** → اسم: **Add New** → `nextjs-headless`
۲. یک رمز مثل `xxxx xxxx xxxx xxxx xxxx xxxx` می‌دهد — ★ همین یک بار نمایش داده می‌شود؛ ذخیره‌اش کن
۳. استفاده با Basic Auth

```
$ curl -u "admin:XXXX XXXX XXXX XXXX XXXX" \
"http://wp-course.local/wp-json/wp/v2/posts?status=draft"
# حالا پیش‌نویس‌ها را هم می‌بینی ←
```

این همان جفت کلید/token دوره‌های قبلیت است: مثل PAT گیت‌هاب – با قابلیت لغو مستقل. در Next.js در `.env` می‌گذاری و هرگز به کلاینت نمی‌روی.

۹.۵ – نوشتن با API (تست)

```
$ curl -X POST -u "admin:XXXX XXXX..." \
-H "Content-Type: application/json" \
-d '{"title": "پست از API!", "status": "draft"}' \
"http://wp-course.local/wp-json/wp/v2/posts"
```

برو پنل – پیش‌نویس جدید! (در پروژه‌های Headless معمولاً از این برای فرم‌ها استفاده می‌شود – فصل ۱۲).

۹.۶ – محدودیت‌های REST (و چرا GraphQL می‌آیم!)

REST وردپرس خوبه ولی سه دردسر داره:

مشکل	مثال	نتیجه
Over-fetch	برای لیست کارتها فقط <code>title</code> و <code>featured_media</code> لازم داری؛ ولی <code>content</code> کامل HTML هر پست هم می‌آید	پهنای باند هدر
N+1 برای روابط	<code>featured_media</code> فقط آی‌دی است؛ عکس واقعی = درخواست جدا برای هر پست	۱۲ پست = ۱۲ درخواست اضافه
embedded پیچیده	راه‌حل <code>_embed</code> هست ولی خروجی را شلوغ و غیرقابل پیش‌بینی می‌کند	DX پایین

GraphQL دقیقاً این سه را حل می‌کند: هرچی خواستی می‌پرسی (نه بیشتر)، روابط در همان کوئری می‌آیند، شکل پاسخ = شکل کوئری. فصل بعد!

جمع‌بندی فصل ✓

- `/wp-json/wp/v2/{نوع}` – بدون هیچ نصبی؛ CPT تو با `show_in_rest` آمده
- `slug` = کلید routing؛ HTML = `content.rendered` آماده؛ ACF در `acf`
- فیلترها: `search` , `orderby` , `per_page` , `?slug=` ؛ هدر `X-WP-Total`

- نوشتن و پیش‌نویس‌ها = **Application Password** (مثل PAT – در env.)
- در سرهای REST → دلیل ورود ما به GraphQL

تمرین فصل ۹

1. سه endpoint را در مرورگر باز کن: JSON – posts، pages، projects را مقایسه کن.
2. با curl پست `my-first-post` را با فیلتر slug بگیر و `id` و `featured_media` را استخراج کن (با python یا فقط چشم!).
3. هدر `X-WP-Total` را ببین: `curl -sI ".../wp/v2/posts?per_page=5" | grep -i x-wp` – چند پست داری؟
4. `Application Password` بساز و پیش‌نویس‌ها را با آن ببین؛ بدون رمز چه خطایی می‌گیری؟ (401!)
5. با curl یک پیش‌نویس جدید بساز و در پنل ببینش.
6. فیلدهای `acf` پروژه‌هایت را در خروجی چک کن (پلاگین ACF to REST API فعال باشد).

▼ نکته تمرین ۲

```
curl -s "http://wp-course.local/wp-json/wp/v2/posts?slug=my-first-post" | python -m json.tool
# [ { "id": 12, "featured_media": 34, ... } ]
```

آرایه با حداکثر یک عضو –

الگوی مهم: اول با slug آیدی بگیر، بعد با آیدی کار کن.

→ **فصل بعد: WPGraphQL** – کوئری دقیق، روابط تو در تو، و پایه‌ی پروژه Next.js!

فصل ۱۰ – WPGraphQL: کوئری دقیق از وردپرس ★

🎯 هدف: پلاگین WPGraphQL را نصب می‌کنیم تا وردپرس یک سرور GraphQL کامل شود. با GraphQL کوئری می‌نویسی: لیست پست‌ها با عکس، CPT با فیلدهای ACF، منوها – همه در یک درخواست. این ستون فقرات پروژه Next.js فصل بعد است.

۱۰.۱ – GraphQL در ۶۰ ثانیه (برای تو که می‌دونی)

```
سرور جواب می‌دهد (همان شکل)
{
  posts {
    nodes {
      title
      featuredImage { node { sourceUrl } }
    }
  }
}
```

```
: تو می‌پرسی (شکل دلخواه):
{
  "posts": {
    "nodes": [
      { "title": "...", },
      { "featuredImage": { ... } }
    ]
  }
}
```

هرچی بخواهی می‌گیری، نه بیشتر – و روابط (عکس پست، نویسنده، دسته‌ها) در همان درخواست. دقیقاً سه مشکل REST فصل ۹ را حل می‌کند.

۱۰.۲ – نصب و GraphQL

1. Plugins → Add New → **WPGraphQL** → Install → Activate

2. در پنل، منوی جدید **GraphQL → GraphQL IDE** را باز کن – یک playground آماده (مثل Swagger/Postman) برای GraphQL

اولین کوئری‌ات را بزن:

```
{
  posts(first: 3) {
    nodes {
      id
      slug
      title
      date
    }
  }
}
```

Ctrl+Enter → خروجی JSON با دقیقاً همین شکل. نوار سمت راست GraphQL، **schema** کامل را نشان می‌دهد (Docs) – مستندات زنده خود وردپرس!

۱۰.۳ – کوئری لیست: پست با عکس شاخص در یک درخواست

همان چیزی که در REST به N+1 دردرس افتاد:

```
{
  posts(first: 6) {
    pageInfo {
      hasNextPage
      endCursor
    }
    nodes {
      id
      slug
      title
      date
      excerpt
      featuredImage {
        node {
          sourceUrl
          altText
          mediaDetails { width height }
        }
      }
      categories {
        nodes { name slug }
      }
    }
  }
}
```

همه چیز: پست + عکس + دسته ها – یک درخواست. (`pageInfo` / `endCursor` برای cursor-based pagination در Next.js – مثل فصل ۹ دوره Prisma!)

۱۰.۴ – کوئری تک پست (برای صفحه جزئیات)

```
{
  post(id: "my-first-post", idType: SLUG) {
    id
    slug
    title
    date
    content
    author {
      node { name }
    }
    featuredImage { node { sourceUrl altText } }
  }
}
```

`idType: SLUG` یعنی با slug جستجو کن – دقیقاً چیزی که از `params` روتر Next.js می گیری.

۱۰.۵ – CPT تو + فیلدهای ACF در GraphQL ★

CPT پروژه (فصل ۷) خودکار در schema آمده: `projects`. برای فیلدهای ACF، پلاگین **WPGraphQL for ACF** را نصب / فعال کن (نسخه جدید ACF خودش پشتیبانی دارد – در Field Group هر فیلد «Show in GraphQL» را on کن یا کل

گروه را).

کوئری واقعی پروژه‌ها:

```
{
  projects(first: 10) {
    nodes {
      id
      slug
      title
      date
      featuredImage { node { sourceUrl altText } }
      projectFields {
        year
        demoUrl
        technologies
        gallery {
          nodes { sourceUrl altText }
        }
      }
      techStack {
        nodes { name slug }
      }
    }
  }
}
```

این JSON دقیقاً همون shape ای است که در کامپوننت React می‌خواهی — بدون transform!

۱۰.۶ — منوها و صفحات

```
{
  menus(where: { location: PRIMARY }) {
    nodes {
      name
      menuItems {
        nodes { label url parentId }
      }
    }
  }
  pages(first: 20) {
    nodes { slug title }
  }
}
```

منو و لیست صفحات برای نوار بالای سایت Next.js — در یک درخواست.

نکته	چرا
پیش‌نویس‌ها با auth	WPGraphQL احراز هویت را با Application Password (فصل ۹) هم ساپورت می‌کند – برای Preview فصل ۱۲
<code>first: N</code> به جای <code>per_page</code>	cursor-based؛ برای صفحه‌های بی‌نهایت عالی
caching	WPGraphQL با پلاگین‌های کش (مثل WPGraphQL Smart Cache) بهتر می‌شود – برای production
secure endpoint	شما endpoint <code>http://wp-course.local/graphql</code> است – همان را Next.js صدا می‌زند

جمع‌بندی فصل

- **WPGraphQL** = سرور GraphQL روی وردپرس؛ playground = GraphQL + مستندات زنده
- لیست با عکس و دسته در یک کوئری؛ تک‌پست با `idType: SLUG`
- CPT = تو `projects` (root field)؛ ACF بعد از فعال‌سازی GraphQL برای ACF در گروه `projectFields`
- منوها و صفحات هم در schema – کل داده سایت در یک API مدرن

تمرین فصل ۱۰ (خروجی هر کوئری = مصرف مستقیم در فصل ۱۱!)

1. WPGraphQL را نصب/فعال کن و اولین کوئری posts را در GraphQL بزن.
2. کوئری لیست با `featuredImage` را اجرا کن – `sourceUrl` عکس‌ها را ببین.
3. کوئری تک‌پست با SLUG را برای `my-first-post` بزن.
4. WPGraphQL for ACF را فعال کن و کوئری `projects` با `projectFields` را بزن – سال و تکنولوژی‌های پروژه‌ها را ببین! (اگر گروه فیلدها نیامد: در ویرایش Field Group، تب GraphQL → Show in GraphQL را on کن.)
5. کوئری منوها را بزن؛ اگر منو خالی بود، در پنل منو را بساز و **Location** اش را Primary کن.
6. (کشف) در GraphQL، پنل Docs را باز کن و root فیلد `mediaItems` را پیدا کن – همه عکس‌های سایت در یک لیست!

▼ نکته تمرین ۴

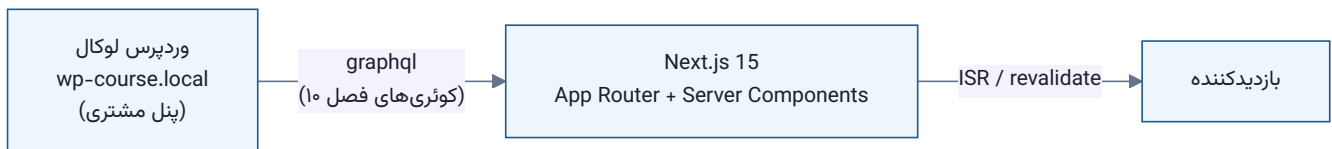
مسیرهای رایج: (الف) ACF نسخه +6: Field Group → Settings → GraphQL → Enable؛ (ب) یا پلاگین «WPGraphQL for ACF». بعد از فعال‌سازی، در GraphQL با Ctrl+Space داخل `projects { ... }` `projectFields` نیامد، schema را رفرش کن (رفرش صفحه).

🚀 **فصل بعد:** لحظه بزرگ – اتصال Next.js به همین API و ساخت سایت واقعی! ➡

فصل ۱۱ – Next.js + وردپرس: پروژه Headless کامل 🚀

🎯 لحظه‌ای که کل دوره برایش ساخته شد: یک اپ Next.js (App Router) که محتوایش را از وردپرس لوکال می‌خواند: لیست پست‌ها با عکس، صفحه جزئیات هر پست با slug، و آرشیو پروژه‌های CPT با فیلدهای ACF. فرانت = تو؛ بک‌اند = وردپرس.

۱۱.۱ – معماری نهایی



و یک نکته امنیتی از دوره‌های قبل: فقط سرور Next.js به وردپرس وصل می‌شود (Server Components) – endpoint وردپرس هرگز به مرورگر لو نمی‌رود.

۱۱.۲ – ست‌آپ پروژه

```
$ npx create-next-app@latest wp-headless --typescript --app --src-dir
$ cd wp-headless
$ npm i graphql-request
```

(`graphql-request` = کلاینت سبک کوئری GraphQL؛ می‌شد با `fetch` خالی هم رفت – پایین‌تر هر دو را می‌بینی.)

`.env.local` (از دوره‌های قبل: هرگز در `git`!):

```
WORDPRESS_API_URL="http://wp-course.local/graphql"
```

⚠️ اگر بعداً روی Vercel دپلوی کردی، `wp-course.local` فقط روی سیستم تو معناست – باید وردپرس روی یک هاست عمومی باشد (فصل ۱۲). لوکال را کامل کن، بعد دپلوی.

لایه داده: `src/lib/wp.ts` – همه کوئری‌های وردپرس یک‌جا:

```

import { GraphQLClient } from "graphql-request";

const client = new GraphQLClient(process.env.WORDPRESS_API_URL!);

type Post = {
  id: string;
  slug: string;
  title: string;
  date: string;
  excerpt?: string;
  content?: string;
  featuredImage?: { node?: { sourceUrl?: string; altText?: string } } };

export async function getPosts(first = 6): Promise<Post[]> {
  const data = await client.request<{ posts: { nodes: Post[] } }>(`
    query GetPosts($first: Int!) {
      posts(first: $first) {
        nodes {
          id slug title date excerpt
          featuredImage { node { sourceUrl altText } }
        }
      }
    `, { first });
  return data.posts.nodes;
}

export async function getPostBySlug(slug: string): Promise<Post | null> {
  const data = await client.request<{ post: Post | null }>(`
    query GetPost($slug: ID!) {
      post(id: $slug, idType: SLUG) {
        id slug title date content
        featuredImage { node { sourceUrl altText } }
      }
    `, { slug });
  return data.post;
}

```

💡 تایپ‌ها دستی نوشتیم؛ ابزار GraphQL Code Generator می‌تواند از schema وردپرس تایپ بسازد (یادت که هست – قدم بعدی حرفه‌ای‌ات!).

۱۱.۳ – صفحه لیست پست‌ها

: src/app/blog/page.tsx

```

import Link from "next/link";
import { getPosts } from "@lib/wp";

export const revalidate = 60; // ISR: (پایین‌تر) کش ۶۰ ثانیه

export default async function BlogPage() {
  const posts = await getPosts(12);

  return (
    <main dir="rtl" className="max-w-5xl mx-auto p-6">
      <h1 className="text-3xl font-bold my-8">بلاگ</h1>
      <div className="grid grid-cols-1 sm:grid-cols-3 gap-6">
        {posts.map((post) => (
          <article key={post.id} className="border rounded-xl overflow-hidden hover:shadow-md">
            {post.featuredImage?.node?.sourceUrl && (
              <img src={post.featuredImage.node.sourceUrl}
                alt={post.featuredImage.node.altText ?? ""}
                className="w-full h-40 object-cover" />
            )}
            <div className="p-4">
              <h2 className="font-semibold">{post.title}</h2>
              <time className="text-sm text-gray-500">
                {new Date(post.date).toLocaleDateString("fa-IR")}
              </time>
              <Link href={`/${blog}/${post.slug}`}
                className="text-blue-600 text-sm mt-2 inline-block">
                ادامه مطلب →
              </Link>
            </div>
          </article>
        ))}
      </div>
    </main>
  );
}

```

Server Component معمولی — `await` مستقیم، بدون `useEffect`. حمله کن به `http://localhost:3000/blog` !

۱۱.۴ — صفحه جزئیات با slug (دینامیک روت)

: `src/app/blog/[slug]/page.tsx`

```
import { notFound } from "next/navigation";
import { getPostBySlug, getPosts } from "@lib/wp";

export const revalidate = 60;

// build (Static Generation): پیش‌ساخت صفحات محبوب در
export async function generateStaticParams() {
  const posts = await getPosts(10);
  return posts.map((p) => ({ slug: p.slug }));
}

export default async function PostPage({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;
  const post = await getPostBySlug(slug);

  if (!post) notFound();

  return (
    <main dir="rtl" className="max-w-2xl mx-auto p-6">
      <h1 className="text-3xl font-bold my-6">{post.title}</h1>
      {post.featuredImage?.node?.sourceUrl && (
        <img src={post.featuredImage.node.sourceUrl}
          alt={post.featuredImage.node.altText ?? ""}
          className="rounded-xl w-full" />
      )}
      </* content از وردپرس HTML آماده است: */>
      <article
        className="prose prose-lg mt-6 max-w-none"
        dangerouslySetInnerHTML={{ __html: post.content ?? "" }}
      />
    </main>
  );
}
```

سه نکته‌ی این فایل که «همه‌چیز» است:

1. `generateStaticParams` = در `build`، برای پست‌های شناخته‌شده صفحه HTML آماده می‌سازد (SSG) – سرعت نور
2. `revalidate = 60` = ISR: بعد از ۶۰ ثانیه، بعدی که آمد، صفحه از نو از وردپرس رندر می‌شود – یعنی مشتری پست جدید می‌گذارد و حداکثر تا یک دقیقه در سایتت ظاهر می‌شود، بدون دیپلوی مجدد!
3. `dangerouslySetInnerHTML` = محتوای HTML گوتنبرگ. امن است چون محتوا از پنل خودت می‌آید (نه کاربران ناشناس) – و اسمش فقط ترسناکه!

۱۱.۵ – آرشیو پروژه‌ها (CPT + ACF)

– با فیلدهای ACF فصل ۱۰: `src/app/projects/page.tsx`

```

type Project = {
  id: string; slug: string; title: string;
  featuredImage?: { node?: { sourceUrl?: string } };
  projectFields?: {
    year?: number;
    demoUrl?: string;
    technologies?: string[];
  };
};

async function getProjects(): Promise<Project[]> {
  const res = await fetch(process.env.WORDPRESS_API_URL!, {
    method: "POST", // GraphQL به POST
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      query: `query { projects(first: 20) {
        nodes { id slug title
          featuredImage { node { sourceUrl } }
          projectFields { year demoUrl technologies }
        } } }`,
    }),
    next: { revalidate: 60 },
  });
  const { data } = await res.json();
  return data.projects.nodes;
}

export default async function ProjectsPage() {
  const projects = await getProjects();
  return (
    <main dir="rtl" className="max-w-5xl mx-auto p-6">
      <h1 className="text-3xl font-bold my-8">پورتفولیو</h1>
      <div className="grid sm:grid-cols-2 gap-6">
        {projects.map((p) => (
          <div key={p.id} className="border rounded-xl p-5">
            <h2 className="font-bold">{p.title}</h2>
            <p className="text-gray-500 text-sm">
              {p.projectFields?.year} {p.projectFields?.technologies?.join(", ")}
            </p>
            {p.projectFields?.demoUrl && (
              <a href={p.projectFields.demoUrl} target="_blank"
                className="text-blue-600 text-sm">دمو {p.projectFields?.demoUrl}</a>
            )}
          </div>
        ))}
      </div>
    </main>
  );
}

```

(اینجا عمده با `fetch` خالی نوشتم تا هر دو روش را دیده باشی – `graphql-request` یا `fetch` خام، سلیقه‌ای است.)

۱۱.۶ – خلاصه چرخه کامل

```

# وردپرس (فصل‌های ۲ تا ۱۰):
docker compose up -d # ببخشید! mean: در Local دکمه Start 🤖

# فرانت:
npm run dev
# /blog ← لیست GraphQL
# /blog/[slug] ← جزئیات
# /projects ← CPT + ACF

```

مشتري در پنل وردپرس پست می‌گذارد → تا ۶۰ ثانیه بعد، سایت Next.js تو به‌روز است. **Headless کامل شد.** 🎉

✓ جمع‌بندی فصل

- `WORDPRESS_API_URL` در `.env.local`؛ لایه داده در `lib/wp.ts` (کوئری‌های فصل ۱۰)
- لیست = `Server Component + grid`؛ جزئیات = `generateStaticParams` + `[slug]` + `notFound`
- `revalidate` = پل زنده بودن محتوا بدون دیپلوی (ISR)
- `content.rendered` با `dangerouslySetInnerHTML` (از پنل خودت = امن)
- `CPT/ACF = typed data` در کامپوننت‌ها

📝 تمرین فصل ۱۱ (پروژه اصلی دوره – حتماً کاملش کن!)

1. پروژه `wp-headless` را بساز، `.env.local` را ست کن و صفحه `/blog` را اجرا کن – پست‌های واقعیات را ببین!
2. صفحه جزئیات `/blog/[slug]` را بساز و از لیست بهش بری.
3. تست ISR: در وردپرس یک پست جدید Publish کن؛ ۶۰ ثانیه صبر، رفرش – هست؟ بعد `revalidate = 10` کن و دوباره تست کن.
4. صفحه `/projects` را با ACF بساز – فیلدهای سال و تکنولوژی را نمایش بده.
5. چالش UI: کارت‌ها را با Tailwind حرفه‌ای کن (سایه، hover، تاریخ فارسی با `toLocaleDateString("fa-IR")`).
6. چالش بعدی: صفحه 404 سفارشی (`not-found.tsx`) و لینک «بازگشت به بلاگ».

▼ نکته تمرین ۳

ISR با `revalidate` کش سرور Next.js را منقضی می‌کند. برای «آنی» شدن (بدون انتظار ۶۰ ثانیه) راه پیشرفته: Webhook از وردپرس به `revalidatePath` در Next.js (Route Handler) – در فصل ۱۲!

➔ **فصل بعد:** حرفه‌ای‌سازی – Preview پیش‌نویس‌ها، فرم‌ها، سئوی Yoast، تصاویر next/image و دیپلوی روی Vercel.

فصل ۱۲ – Headless پیشرفته: Preview، فرم‌ها، SEO، تصاویر و دپلوی

🎯 هدف: فصل «تبدیل دمو به محصول» – چهار قابلیت که پروژه Headless واقعی را از دمو جدا می‌کند: پیش‌نویس‌بینی برای مشتری (Preview)، فرم‌ها، سئوی کامل با داده Yoast، تصاویر بهینه next/image و دپلوی روی Vercel.

۱۲.۱ – Preview: مشتری پیش‌نویس را ببیند! ★

مشکل: پست پیش‌نویس از API عمومی نمی‌آید (فصل ۹). مشتری می‌خواهد قبل از Publish ببیند چطور شده. راه‌حل: **Preview Mode** – یک Route Handler که با secret وردپرس، درخواست preview را با احراز هویت رندر می‌کند.

گام ۱ – `src/app/api/preview/route.ts` :Route Handler

```
import { draftMode } from "next/headers";
import { redirect } from "next/navigation";
import { NextRequest } from "next/server";

export async function GET(req: NextRequest) {
  const secret = req.nextUrl.searchParams.get("secret");
  const slug = req.nextUrl.searchParams.get("slug") ?? "";

  // چک کن - هرکس با لینک تصادفی وارد نشود .env سیکرت را با
  if (secret !== process.env.PREVIEW_SECRET) {
    return new Response("Invalid secret", { status: 401 });
  }

  (await draftMode()).enable(); // فعال‌سازی Draft Mode در Next.js
  redirect(`/blog/${slug}`);
}
```

گام ۲ – خواندن پیش‌نویس در `lib/wp.ts`: وقتی draft mode روشن است، کوئری با احراز هویت و `asPreview` بزن:

```
export async function getPreviewPost(slug: string, token: string) {
  const client = new GraphQLClient(process.env.WORDPRESS_API_URL!, {
    headers: { Authorization: `Basic ${Buffer.from(
      `${process.env.WP_USER}:${process.env.WP_APP_PASSWORD}`
    ).toString("base64")}` },
  });
  const data = await client.request(`
    query($slug: ID!) {
      post(id: $slug, idType: SLUG, asPreview: true) {
        slug title content date
      }
    }
  `, { slug });
  return data.post;
}
```

(user:password از Base64 — فصل ۹ همان WP_APP_PASSWORD)

گام ۳ — استفاده: در صفحه [slug] ، وقتی `draftMode().isEnabled` است، به جای `getPostBySlug` ، پیش‌نویس را با توکن بخوان.

گام ۴ — لینک در وردپرس: دکمه پیش‌نمایش وردپرس را طوری شخصی‌سازی کن (یا دستی) به:

```
https://your-site.vercel.app/api/preview?secret=XXX&slug=my-draft
```

مشتری روی «Preview» می‌زند → نسخه پیش‌نویس را روی سایت واقعی Next.js می‌بیند — بدون publish! (پلاگین‌هایی مثل Faust.js/Headlessooks همین جریان را خودکار می‌کنند.)

۱۲.۲ — فرم‌ها: تماس با ما و کامنت‌ها

فرم تماس (ساده‌ترین راه درست): فرم POST → Server Action → React به REST وردپرس (فصل ۹ یادته) به یک CPT اختصاصی مثل پیام‌ها :

```
"use server";
export async function submitContact(formData: FormData) {
  const res = await fetch(`${process.env.WORDPRESS_API_URL!.replace("/graphql", "")}/wp/v2/messages
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Basic ${Buffer.from(
        `${process.env.WP_USER}:${process.env.WP_APP_PASSWORD}`
      ).toString("base64")}`,
    },
    body: JSON.stringify({
      title: String(formData.get("subject")),
      status: "private", // پیام‌ها فقط در پنل دیده شوند
      acf: { email: formData.get("email"), message: formData.get("message") }, // ACF to REST
    }),
  });
  if (!res.ok) return { error: "ارسال نشد" };
  return { ok: true }; // مشتری در پنل وردپرس، باکس «پیام‌ها» را می‌بیند
}
```

الگوی زیبا: پیام‌ها = یک CPT با ACF؛ مشتری همان‌جا در پنل وردپرس می‌بیندشون. (برای اسپم: honeypot + Google reCAPTCHA سمت کلاینت).

کامنت‌ها: دو راه – (۱) POST به `/wp/v2/comments` (باید `allow_comments` باشد؛ اسپم را Akismet می‌گیرد)، (۲) سرویس آماده مثل Giscus (بر پایه GitHub Discussions – برای بلاگ‌های فنی محبوب).

۱۲.۳ – SEO: داده metadata → Yoast نیکست

Yoast (فصل ۵) مقادیر سئو را در وردپرس می‌گذارد؛ با WPGraphQL for SEO (افزونه یوست برای GraphQL) یا REST `/wp/v2/posts?context=edit` یا پلاگین WPGraphQL Yoast SEO می‌آیند. الگوی مصرف در Next.js:

```
export async function generateMetadata({ params }): Promise<Metadata> {
  const { slug } = await params;
  const post = await getPostBySlug(slug);
  return {
    title: post?.title,
    description: post?.excerpt?.replace(/<[^>+>/g, "").slice(0, 150),
    openGraph: {
      title: post?.title,
      images: post?.featuredImage?.node?.sourceUrl ? [post.featuredImage.node.sourceUrl] : [],
    },
  };
}
```

و دو فایل استاندارد: `sitemap.ts` (همه slug ها از وردپرس) و `robots.ts` – با توابع آماده Next.js. (اسکلت‌شان را با `sitemap.ts` سرچ کن در داکيومنت؛ داده‌اش همین کوئری لیست پست‌هاست).

۱۲.۴ – تصاویر: next/image با رسانه وردپرس

`<img>` ساده کار می‌کند ولی `next/image` بهینه‌تره (lazy load، سایز درست). برای اینکه Next.js عکس‌های دامنه وردپرسی را قبول کند:

```
// next.config.ts
const nextConfig = {
  images: {
    remotePatterns: [
      { protocol: "http", hostname: "wp-course.local" },
      // بعداً production و دامنه وردپرس
    ],
  },
};
```

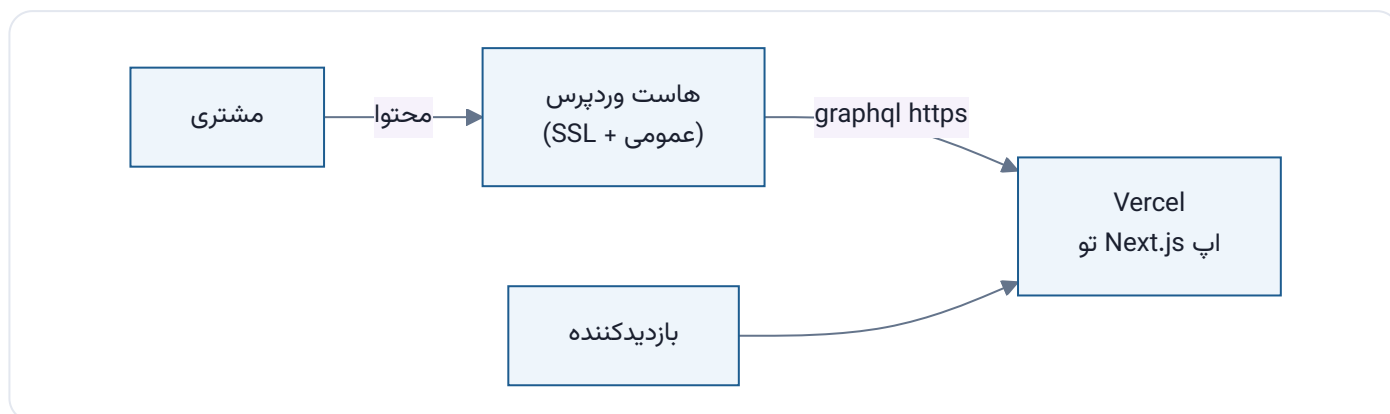
و در کامپوننت:

```
<Image src={p.featuredImage.node.sourceUrl} alt={p.featuredImage.node.altText ?? ""}
width={800} height={450} className="rounded-xl" />
```

(اندازه‌های `mediaDetails` فصل ۱۰ را می‌توانی بدهی تا layout shift صفر شود).

۱۲.۵ – دپلوی: Vercel + وردپرس عمومی

معماری production:



گام‌ها:

1. **وردپرس عمومی:** یک هاست وردپرسی (یا VPS با داکر – دوره داکرت!) – سایت وردپرس فقط برای پنل و API است؛

تمش مهم نیست، پیش‌فرض کافی است

2. **تست:** `https://دامنه/wp-json/graphql` و `/graphql` از اینترنت در دسترس باشد

3. **Vercel:** ریپو Next.js را import کن → `WORDPRESS_API_URL` Environment Variables: (دامنه واقعی) + رازهای

preview

4. **تحويل بگير:** هر push به main = دپلوی؛ هر تغيير محتوا در وردپرس = تا `revalidate` ثانيه در سایت

بعدش (اختیاری حرفه‌ای): Webhook بلافاصله – پلاگین در وردپرس روی `save_post` یک درخواست به

`/api/revalidate?secret=...` بزند و تو در Route Handler با `revalidatePath("/blog")` کش را فوری باطل کنی

– همان الگوی `$transaction` دوره Prisma: به‌روزرسانی اتمی محتوا. (WPGraphQL Smart Cache هم راه GraphQL-native این کار است.)

۱۲.۶ – چک‌لیست تحويل پروژه Headless

- [] وردپرس عمومی با SSL؛ پنل با رمز قوی؛ کاربر مشتری = Editor
- [] CPT/ACF با `GraphQL enabled – versioned` در پلاگین خودت `show_in_rest`
- [] Application Password در env های Vercel (هرگز در git)
- [] ISR/revalidate تنظیم؛ تست «پست جدید تا یک دقیقه می‌آید»
- [] Yoast → `metadata/sitemap/robots`
- [] `next/image + remotePatterns`
- [] فرم تماس + اسپم‌گیر
- [] بکاپ خودکار وردپرس (UpdraftPlus – فصل ۵)

✓ جمع‌بندی فصل

- **Preview** = draft mode + secret + احراز هویت Application Password – مشتری پیش‌نویس را روی سایت واقعی می‌بیند
- فرم‌ها: CPT → REST POST → Server Action → پیام‌ها؛ مشتری در پنل می‌بیند
- SEO: Yoast → `generateMetadata` + sitemap/robots
- تصاویر: `remotePatterns` + next/image + alt از رسانه
- دپلوی: وردپرس عمومی + ISR + Vercel؛ ارتقا: Webhook revalidate فوری

📝 تمرین فصل ۱۲ (فینال پروژه!)

1. Preview Mode را کامل راه بینداز: secret در هر دو طرف + Route Handler + تست با یک پیش‌نویس.
2. فرم تماس بساز (Client Component با `useActionState` فصل ۱۰ دوره Next) که پیام را به `messages` CPT بفرستد – در پنل وردپرس ببینش.
3. `generateMetadata` را به صفحه جزئیات اضافه کن – منبع صفحه را ببین (عنوان سئو یوست اگر وصل کردی).
4. همه `<img>` ها را به `next/image` مهاجرت کن + `remotePatterns`.
5. (اگر هاست داری 🌟) وردپرس را عمومی کن و پروژه را روی Vercel دپلوی کن – لینکش را به رزومه‌ات اضافه کن!
6. چالش فینال: Webhook revalidate – پلاگین WP Webhooks یا کد `save_post` فصل ۸ که به Route Handler تویی curl بزند.

▼ اسکلت تمرین ۶

```
// app/api/revalidate/route.ts
import { revalidatePath } from "next/cache";
import { NextRequest } from "next/server";

export async function POST(req: NextRequest) {
  if (req.nextUrl.searchParams.get("secret") !== process.env.PREVIEW_SECRET) {
    return new Response("no", { status: 401 });
  }
  revalidatePath("/blog");
  revalidatePath("/projects");
  return Response.json({ revalidated: true });
}
```

و در پلاگین PHP فصل ۸: `wp_remote_post("https://site.vercel.app/api/revalidate?secret=...",`

`save_post` روی `[...]` . حالا publish = به‌روزرسانی فوری! 🚀

[→ فصل آخر: چیت‌شیت جامع و گلاساری – مرور سریع کل دوره.](#)

فصل ۱۳ – چیت‌شیت، گلاساری و نقشه راه

🎯 هدف: مرجع مرور سریع دو دنیا: مدیریت وردپرس + توسعه Headless.

۱۳.۱ – چیت‌شیت پنل وردپرس

— مسیره‌های پنل —

Posts / Pages	← محتوای زمان‌دار / ثابت (فصل ۳)
Appearance → Themes	← نصب و فعال‌سازی تم
Appearance → Editor	← Site Editor (تم‌های بلوکی)
Plugins → Add New	← Install ≠ Activate! نصب پلاگین
Users → Profile	← Application Passwords (!پایین صفحه)
Settings → General	← Timezone، زبان (فارسی!)
Settings → Permalinks	← CPT بعد از ساخت Post name + Save ★
Settings → Reading	← صفحه اصلی / تعداد پست

— WP-CLI (در Local → Open Site Shell) —

```
wp core version
wp plugin list / install x --activate / update --all / delete x
wp theme list / activate x
wp scaffold child-theme x-child --parent=x
wp post list --post-type=project --fields=ID,post_title
wp search-replace "http://old" "https://new" # مهاجرت دامنه!
```

— نجات —

Local Snapshot	← کامیت محیط (قبل از هر تغییر بزرگ)
Recovery mode	← اگر پلاگینی سایت را شکست، ایمیل لینک ریکاوری می‌آید
rename plugins folder	← پوشه پلاگین مقصر را تغییر نام بده، FTP/Explorer: آخرین راه

```
# — GraphQL (۱۰ فصل) —
query GetPosts($first: Int!) {
  posts(first: $first) {
    pageInfo { hasNextPage endCursor }
    nodes {
      id slug title date excerpt
      featuredImage { node { sourceUrl altText } }
      categories { nodes { name slug } }
    }
  }
}

query GetPost($slug: ID!) {
  post(id: $slug, idType: SLUG) {
    title content date
    author { node { name } }
    featuredImage { node { sourceUrl } }
  }
}

query GetProjects {
  projects(first: 20) {
    nodes {
      slug title
      featuredImage { node { sourceUrl } }
      projectFields { year demoUrl technologies } # ACF
      techStack { nodes { name } } # taxonomy
    }
  }
}
```

```
// — Next.js الگوهای (۱۲-۱۱ فصل) —
export const revalidate = 60; // محتوا تا ۶۰ ثانیه تازه - ISR
export async function generateStaticParams() { ... } // SSG ها slug برای
const { slug } = await params; // params یک Promise است!
if (!post) notFound(); // تمیز 404
dangerouslySetInnerHTML={{ __html: post.content }} // HTML (امن = خودت) گوتنبرگ (از پنل خودت = امن)
(await draftMode()).enable(); // Preview Mode
revalidatePath("/blog"); // وردپرس → به‌روزرسانی فوری Webhook
```

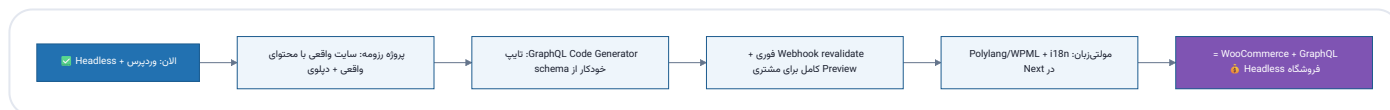
```
# — REST (۹ فصل) سریع —
GET /wp-json/wp/v2/posts?slug=xxx&per_page=10
GET /wp-json/wp/v2/projects # CPT (show_in_rest!)
POST /wp-json/wp/v2/posts -u "user:app-pass" -d '{"title":"...", "status":"draft"}'
هدر X-WP-Total # تعداد کل (pagination)
```

انگلیسی	فارسی	یک جمله
CMS	سامانه مدیریت محتوا	پنلی که مشتری با آن محتوا می‌گذارد
Post / Page	نوشته / برگه	محتوای زمان‌دار / ثابت
Gutenberg / Block	گوتنبرگ / بلوک	ادیتور و واحد UI (با React!)
Theme (Block/Classic)	تم بلوکی/کلاسیک	ظاهر سایت – جدید: Site Editor
Child Theme	تم فرزند	لایه تغییرات تو، جدا از تم اصلی
Plugin	افزونه	بسته قابلیت (npm وردپرس)
Role	نقش کاربری	Subscriber→Author→Editor→Admin
Slug	نامک	بخش آخر URL – کلید routing تو
CPT (Custom Post Type)	نوع محتوای سفارشی	محصول/پروژه/نمونه کار – ساختار داده
Custom Field / ACF	فیلد سفارشی	فیلدهای ساخت‌یافته (...year, url)
Taxonomy	طبقه‌بندی	دسته/برچسب سفارشی برای فیلتر
WP-CLI	–	خط فرمان وردپرس
Hook (Action/Filter)	قلاب	رویداد / تغییر مقدار – هسته توسعه PHP
WP REST API	–	JSON – wp-json/wp/v2/... استاندارد
Application Password	رمز برنامه	توکن برای API (مثل PAT)
WPGraphQL / GraphQL	–	سرور و پلی‌گراند GraphQL وردپرس
Headless	بدون سر (فرانت جدا)	وردپرس فقط بک‌اند؛ فرانت = Next.js
ISR / revalidate	بازاعتبارسنجی	کش با انقضا – محتوا تازه بدون دیپلوی
Draft Mode	حالت پیش‌نویس	دیدن پیش‌نویس در سایت Next.js

۱۳.۴ – ده گیر کلاسیک و علاج

مشکل	علاج
CPT در پنل نیامد	کد را چک کن؛ بعد Settings → Permalinks → Save (رفرش قواعد)
CPT در REST 404 داد	<code>show_in_rest ⇒ true</code> نداشتی (فصل ۷)
فیلدهای ACF در API نیست	پلاگین ACF to REST / Show in GraphQL در Field Group
کوئری GraphQL فیلد را نمی‌شناسد	refresh :GraphQL = autocomplete schema در Ctrl+Space
عکس در next/image لود نشد	remotePatterns در next.config (فصل ۱۲)
محتوا آپدیت نمی‌شود	<code>revalidate</code> را چک کن؛ برای فوری: Webhook (فصل ۱۲)
Preview خالی/404	secret دو طرف یکی نیست؛ Application Password فعال؟
سایت سفید شد (کد PHP)	WP_DEBUG روشن کن؛ آخرین پلاگین/اسنیپت را خاموش؛ Snapshot بازگردان
ارور حافظه/سنگینی	پلاگین‌های اضافه را کم کن؛ PHP memory در Local قابل افزایش
<code>params</code> بدون await ارور داد	در Next 15 params یک Promise است: <code>await params</code>

۱۳.۵ – نقشه راه پس از این دوره



سه توصیه پایانی:

1. **پروژه رزومه بساز:** «سایت Headless وردپرس + Next.js» با داده واقعی و دپلوی Vercel – در بازار ایران این ترکیب کمیاب و پول‌ساز است؛ لینکش را بالای رزومه‌ات بگذار.
2. **وردپرس را مثل محصول نگه دار:** آپدیت‌ها، بکاپ خودکار، پلاگین کم، رمز قوی – ۹۰٪ ماجرا همین انضباط است.
3. **داکیومنت رسمی:** (REST + Block Editor) developer.wordpress.org و www.wpgraphql.com/docs – هر دو با پایه‌ای که الان داری خوانا شده‌اند.

سفر پنج‌گانه‌ات تا اینجا: ☒ Docker → ☒ CLI → ☒ Git/GitHub → ☒ PostgreSQL+Prisma ☒ WordPress+Headless

تو الان می‌توانی: دیتابیس طراحی کنی، با تیم کار کنی، ترمینال را هدایت کنی، روی سرور دپلوی کنی – و حالا هم CMS برای مشتری داری هم فرانت مدرن. پکیج کامل فول‌استک! 🚀🎉